Republic of Tunisia
Ministry of High Education and Scientific Research
***
University of Monastir
High Institute of Computer Science and Mathematics of Monastir
Department of Technology
***

Graduation Project

In order to obtain the
National Diploma of Engineering in Applied Sciences and Technology

Major:
Electronics: Microelectronics

Presented by:

Najd Elaoud and Selim Triki

Design and development of the IOBOX-BWS industrial platform for data acquisition, processing, and control

---

Presented *X* **July** 2026 **in front of the jury members :**

Mrs. UNKOWN	President
Mr. UNKOWN	Lecturer
Mr. Abdessalem Ben Abdelali	Academic Supervisor
Mr. Jamel Hmidi	Industrial Supervisor

Academic year: 2025/2026

ACKNOWLEDGEMENTS

I am humbled and grateful to all who have helped me translate these ideas into something concrete that goes far beyond a simple level.

I would like to express my sincere gratitude to my academic supervisor, **Mr. Abdessalem BEN ABDELALI**, for his guidance throughout this project, for reviewing my work, and for his valuable remarks and suggestions that helped improve the quality of this report.

I would also like to express my deepest appreciation to my industrial supervisor, **Mr. Jamel HMIDI**, for his continuous support, technical guidance, and availability throughout the internship. His expertise, advice, and encouragement were invaluable to the successful completion of this project.

Last but not least, I am deeply grateful to all the jury members **Mr. UNKOWN** and **Mrs. UNKOWN** for agreeing to evaluate this work.

Without your support, this accomplishment would not have been achieved. I really appreciate it.

DEDICATION

To my precious family, I would never be where I am now without you.

To my dear father ***Noureddine****:*
Thank you for your hard work, guidance, and the values you have instilled in me throughout my life. Your sacrifices and dedication have always inspired me to strive for success. May God bless you with health, happiness, and a long life.

To my dear mother ***Soufia****:*
You have done more than a mother can do to ensure that her children follow the right path in their lives and studies. I dedicate this work to you as a testimony of my deep love. May God, the Almighty, preserve you and grant you health, long life, and happiness.

To my dear sister ***Nibras****:*
Thank you for your constant support, encouragement, and affection. Having you by my side has always been a source of strength and motivation.

To my dear Friends, Words can't express how grateful I am to have your friendship in my life. Thank you for supporting me, accepting me, and being wonderful friends.

Lastly, to all the kind people I met during my six years here, to all the institute students, professors, administrators and workers, thank you for being part of my exciting university career.

Najd Elaoud

DEDICATION

*"What you leave behind is not what is engraved in stone monuments, but what is woven into the lives of others." — **Pericles***

*To my dear father **Lotfi**:*
This project and my success in my academic journey could not have been achieved without your presence in my life. I am deeply grateful for the comfort and opportunities you provided for us, and for teaching me that with dedication and perseverance nothing is impossible. I hope that one day I can repay you, even in a small way, for all that you have done.

*To my dear mother **Monia**:*
The love you gave us made us stronger, and the dedication and support you offered whenever we fell down or felt low is the most precious gift in life. Thank you for being the light that guided us and for showing us the strength of unconditional love.

*To my dear sister **Ahlem**:*
Thank you for bringing happiness into my life and for being my partner through both struggles and hard work. Your emotional support has been invaluable, and I am truly grateful for the strength and companionship you have given me.

*To my dear brother **Youssef**:*
I cannot wait to see you thrive and shine in life, overcoming every turbulence along the way. I will always be here as your mentor whenever you need guidance.

*To my dear **friends** ,Thank you for all the support you have given me and for the joy you bring into my life. I am truly grateful that you accept every side of my personality, and I cherish the real human connection we share. Seeing you succeed in life is part of my own dreams, and I look forward to celebrating your achievements alongside you.*

Selim Triki

ABSTRACT

The development of flexible and interoperable industrial platforms is essential for addressing the growing integration challenges of Industrial Internet of Things (IIoT) systems. This report presents the design and development of the IOBOX, a configurable industrial Input/Output platform intended to simplify the acquisition, processing, control, and exchange of data between sensors, actuators, and supervisory systems.

The developed platform is based on an STM32 microcontroller and integrates multiple industrial communication interfaces, peripheral drivers, and sensor management modules within a modular software architecture. The implementation emphasizes hardware abstraction, software reusability, and scalability, enabling efficient integration of heterogeneous devices while reducing development effort.

The resulting solution provides a versatile and reusable platform for industrial monitoring, automation, and IoT applications. By improving interoperability and simplifying system integration, the IOBOX contributes to the development of more efficient and scalable industrial solutions.

Keywords: Industrial IoT, Embedded Systems, STM32, IO-Box, Data Acquisition, Industrial Communication Protocols, Automation, Software Architecture.

RESUME

Le développement de plateformes industrielles flexibles et interopérables est devenu essentiel pour répondre aux défis croissants d'intégration des systèmes de l'Internet Industriel des Objets (IIoT). Ce rapport présente la conception et le développement de l'IOBOX, une plateforme industrielle d'Entrées/Sorties configurable destinée à simplifier l'acquisition, le traitement, le contrôle et l'échange de données entre les capteurs, les actionneurs et les systèmes de supervision.

La plateforme développée repose sur un microcontrôleur STM32 et intègre plusieurs interfaces de communication industrielle, pilotes de périphériques et modules de gestion de capteurs au sein d'une architecture logicielle modulaire. L'implémentation met l'accent sur l'abstraction matérielle, la réutilisabilité logicielle et l'évolutivité, permettant une intégration efficace de dispositifs hétérogènes tout en réduisant les efforts de développement.

La solution obtenue constitue une plateforme polyvalente et réutilisable pour les applications de supervision industrielle, d'automatisation et d'IoT. En améliorant l'interopérabilité et en simplifiant l'intégration des systèmes, l'IOBOX contribue au développement de solutions industrielles plus efficaces et évolutives.

Mots-clés : Internet Industriel des Objets (IIoT), Systèmes Embarqués, STM32, IOBOX, Acquisition de Données, Protocoles de Communication Industrielle, Automatisation.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

GLOSSARY OF ACRONYMS

BSP: Board Support Package

UART : Universal Asynchronous Receiver-Transmitter

LIN: Local Interconnect Network

USB : Universal Serial Bus

CAN : Controller Area Network

I2C : Inter-Integrated Circuit

SPI : Serial Peripheral Interface

DAC : Digital to Analog Converter

ADC : Analog to Digital Converter

PWM : Pulse Width Modulation

DMA : Direct Memory Access

GPIO : General Purpose Input Output

PID : Proportional Integral Derivative

GENERAL INTRODUCTION

Industrial systems are increasingly relying on connected devices capable of acquiring, processing, and exchanging information in real time. In such environments, sensors, actuators, and supervisory systems must interact efficiently through various communication interfaces and protocols. However, integrating these heterogeneous components often requires significant development effort, particularly when flexibility, scalability, and interoperability are required.

To address these challenges, Be Wireless Solutions (BWS) launched the development of the IOBox platform, a configurable industrial Input/Output system designed to serve as a universal interface between field devices and monitoring or control systems. The platform is intended to simplify the integration of sensors and actuators while providing data acquisition, signal processing, control, and communication functionalities within a single solution.

The IOBox supports multiple industrial communication protocols and offers both analog and digital input/output capabilities. Its modular architecture enables adaptation to various industrial applications, ranging from environmental monitoring and energy management to process automation and equipment supervision. By centralizing these functionalities in a single platform, the IOBox reduces deployment complexity and accelerates the development of industrial IoT solutions.

The objective of this end-of-study project is to participate in the design and implementation of the IOBox platform. The work focuses on the development of embedded software components, communication interfaces, peripheral drivers, sensor integration, and system architecture required to ensure reliable operation in industrial environments.

This report, which crowns the work entrusted to us, is structured around three chapters:

Chapter 1 presents the general context of the project, the host company, the problem statement, the project objectives, and the adopted development methodology.

Chapter 2 introduces the general concept of the IOBox platform and describes the hardware components, software tools, communication protocols, sensors, and peripherals used throughout the project.

Chapter 3 details the design and implementation of the platform, including the software architecture, peripheral configuration, driver development, communication management, sensor integration, and system validation.

CHAPTER 1

GENERAL PROJECT CONTEXT

1.1 Introduction

This chapter gives an overview of the professional and academic context of our graduation project. First, it starts by introducing the host company, Be Wireless Solutions (BWS), covering its structure, its line of work, and the main IoT solutions it offers. Then we'll outline the project's requirements, starting with explaining the problem that was identified, then explaining how the IOBOX platform came out of it, the development method used, and the project's goals and the effects of the project.

1.2 Academic Institution

The Higher Institute of Computer Science and Mathematics of the University of Monastir (ISIMM) was created by decree number 1623 on July 9, 2002. It is a public scientific establishment of higher education under the authority of the Ministry of Higher Education and Scientific Research [1].

ISIMM offers engineering courses in Computer Science, Mathematics and Electronics. This graduation project was developed within the Electronics department, specialized in Microelectronics, following the requirements for the National Diploma in Engineering in Applied Sciences and Technology.

1.3 Host Company Presentation

1.3.1 Company Overview

Be Wireless Solutions (BWS) is a technology group founded in 2016, specializing in the development of IoT/AI solutions to optimize the use of resources such as electricity, water, and fuel. Operating across three continents, BWS supports industrial companies, logistics and transportation companies, utilities, and local governments in their transition towards more sustainable and efficient resource management.[2]

Figure 1.1: Be Wireless Solutions (BWS) logo

1.3.2 Organization Chart

BWS has different departments that work together during the whole process of creating industrial and IoT solutions. These departments are management, research and development, hardware design, embedded software development, cloud services, technical support, and business development.

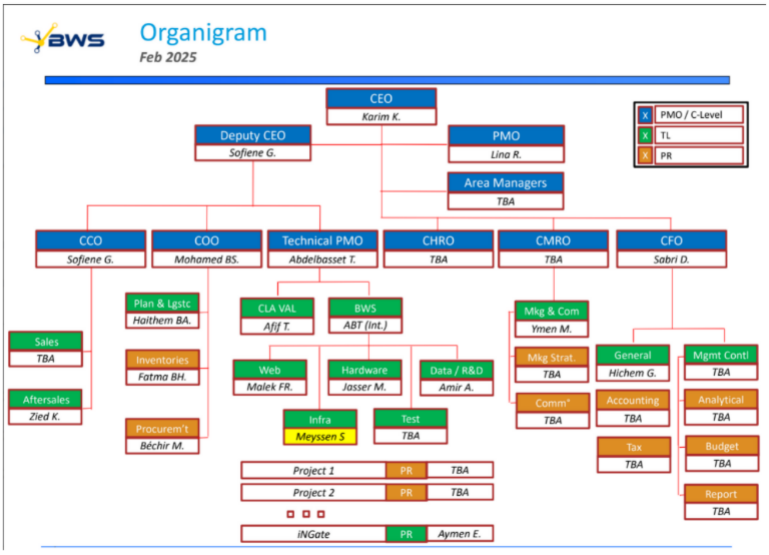

Figure 1.2: Organization chart of Be Wireless Solutions

1.3.3 Main Activities and Services

BWS creates many different industrial IoT solutions that help make operations more efficient, manage resources better, and keep track of processes more effectively.

Energy Monitoring and Management

BWS provides intelligent energy monitoring systems capable of collecting, analyzing, and visualizing electricity consumption data in real time. These solutions help organizations identify energy-intensive equipment, optimize consumption, reduce operational costs, and improve environmental performance.

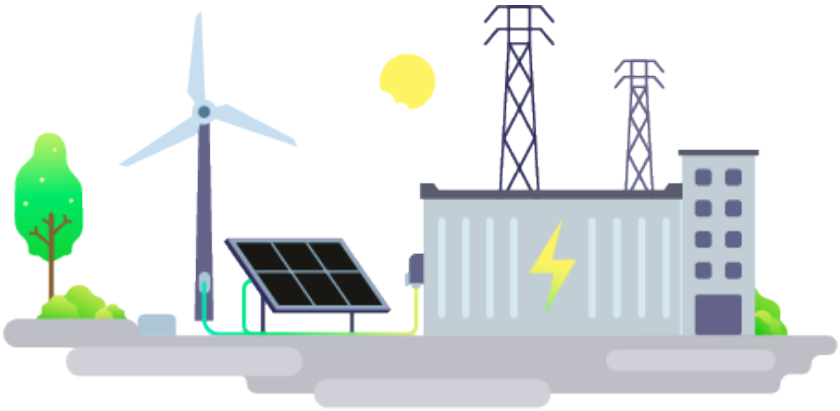

Figure 1.3: Electricity consumption monitoring solution

Water Monitoring Solutions

The company provides smart water management solutions that allow for remote reading of meters, detecting leaks, monitoring unusual water use, and helping to use water resources more efficiently.

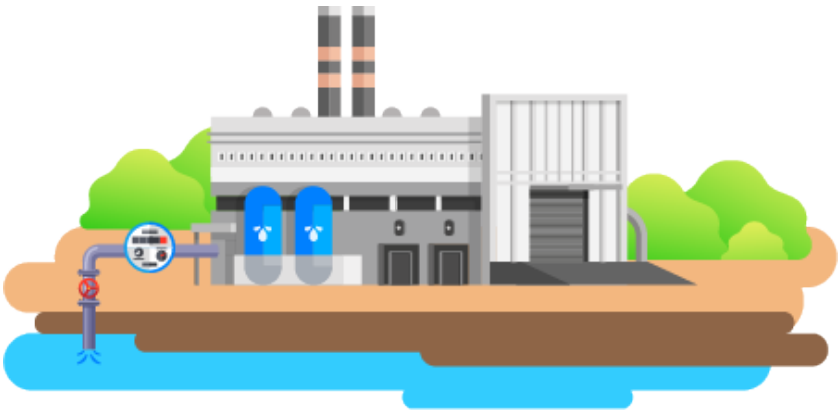

Figure 1.4: Water consumption monitoring solution

Fleet and Fuel Management

BWS develops fleet management systems that provide real-time vehicle tracking, fuel consumption analysis, route optimization, and driver behavior monitoring.

Figure 1.5: Fleet and fuel management solution

Remote Monitoring and Equipment Control

The company provides remote supervision solutions that allow operators to monitor industrial equipment, detect anomalies, generate alerts, and perform remote control operations through dedicated software platforms.

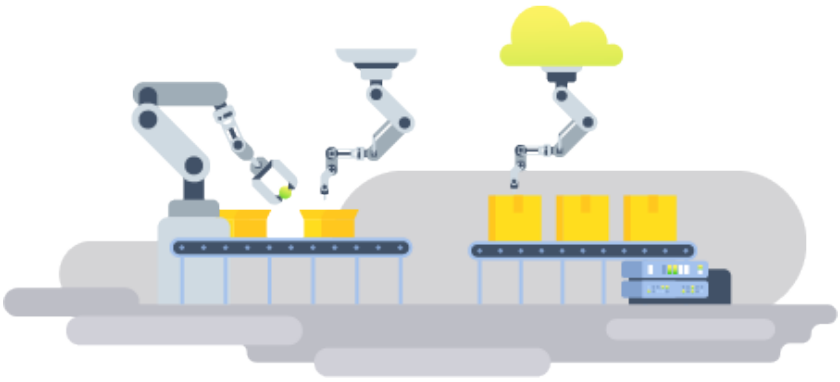

Figure 1.6: Remote equipment monitoring and control

Cold Chain Monitoring

BWS also offers cold-chain monitoring systems intended for warehouses, refrigerated transportation, and pharmaceutical storage facilities. These solutions ensure

compliance with temperature requirements and provide instant alerts when abnormal conditions occur.

Figure 1.7: Cold-chain monitoring solution

1.4 Project Specification

1.4.1 Problem Statement

Industrial IoT systems are usually composed of a multitude of sensors, actuators, and communication interfaces. For each client project BWS developed dedicated firmware for each new setup; every time there was a new set of peripherals, they would need to be reconfigured and the drivers rebuilt. This approach resulted in slower development, limited reuse of code between projects, and made system maintenance harder when hardware changed.

To address these issues, BWS initiated the IOBOX project. The aim was to develop a flexible, hardware-independent platform that can support different setups on one common firmware base. In industrial environments there are often different communication protocols and interface standards. Handling these through custom solutions adds complexity and makes it harder to scale the system.

The goal is to provide a customizable and reusable system for industrial data collection and control. This system aims to make the addition of peripherals easier while keeping the system flexible, easy to relocate between setups, and easier to maintain.

1.4.2 Presentation of the IOBOX Platform

The IOBOX is an industrial flexible system that helps to interconnect sensors, actuators and communication tools in industrial environments.

It adopts a modular approach that distinguishes the parts dependent on specific hardware from the parts that manage the main functions. This is achieved through specific software tools such as drivers, a board support package, and Manager, which facilitate use in different systems and maintain an easy update.

The IOBOX can interface with various types of industrial equipment such as analog and digital inputs and outputs, communication ports and measurement devices. Its software configuration can be adapted to various industrial requirements without major changes in the firmware.

By connecting the hardware and the software in a straightforward way, the IOBOX simplifies the assembly and the use in industrial environments.

1.4.3 Project Objectives

The key objective of this project is to develop the IOBOX platform which will collect, handle, control, and transmit data in real-time.

To be more specific, the project aims to:

- Developing a scalable framework that is capable of acquiring the data and controlling it according to industry needs.
- Provide a unified software abstraction layer for heterogeneous peripherals.
- Delivering programmable analogue and digital input-output features.
- Supporting industrial communication protocols such as Modbus RTU via RS485, CAN, LIN, and multi-channel UART, ensuring the compatibility with other industrial devices.
- Develop a modular software architecture based on abstraction layers and reusable software components.
- Design the software architecture to allow new sensors, actuators, and communication interfaces to be added with minimal modification to the existing code.

The resulting platform will lower development costs and reduce time-to-market by offering a reusable embedded architecture that adapts to various industrial settings.

1.4.4 Development Methodology

The development of this project was based on the V-Model approach, a systematic development framework widely used in embedded and industrial applications. This methodology aligns each development stage with a corresponding validation phase; this

pairing helps maintain traceability throughout the system's lifecycle.

The project activities were organized according to the different phases of the V-Model. The requirements-gathering phase focused on identifying the platform capabilities and technical constraints required by BWS. Once these requirements were established, the system architecture was designed, including the definition of the four-layer software structure and the selection of the hardware components. The detailed design stage then concentrated on the BSP modules, communication manager, and sensor drivers.

After the design work was completed, the implementation phase involved coding BSP drivers, sensor management systems, communication protocols, and application-level functionalities. The project's validation included testing each BSP module individually, verifying sensor data acquisition and Modbus communication in an integrated environment, and performing complete system-wide testing to confirm that all operational requirements were met.

Table 1.1: Application of the V-Model to the Project

V-Model Phase	**Project Activity**
Requirements Analysis	Platform specifications derived from BWS requirements.
System Design	Hardware selection and software architecture definition.
Software Design	BSP API design and manager module interface definition.
Implementation	Development of BSP drivers, sensor managers, Modbus communication, CAN communication, and filtering algorithms.
Unit Testing	Validation of individual BSP modules and drivers.
Integration Testing	Verification of sensor chains and end-to-end Modbus communication.
System Testing	Complete platform validation on the target hardware.

The ascending branch focuses on checking and making sure everything works correctly through unit testing, integration testing, system testing, and final acceptance

testing.

The use of the V-Model helps improve the quality of the software, lowers the risks during development, and makes it easier to manage the project all through its different stages.

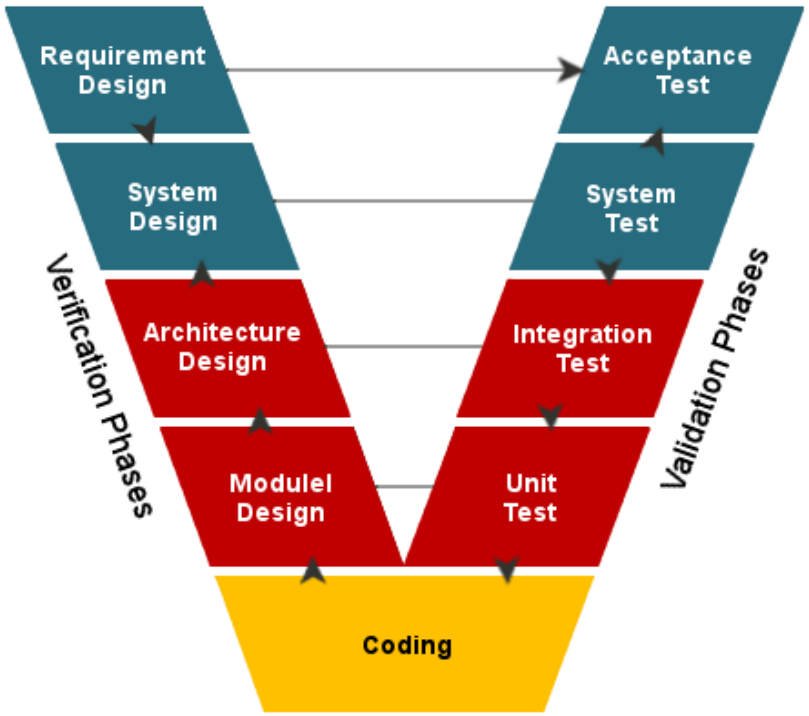

Figure 1.8: V-Model development lifecycle

1.5 Conclusion

This chapter explained the main structure of the project, starting by introducing the academic institution and the hosting company. Next, we presented the project's framework by describing the problem it aims to solve, its objectives, and the requirements that guided it's development. Finally, we outlined the methodology adopted throughout the project.

The next chapter talks about the technical setup of the IOBox platform. It covers the physical parts of the system, the software tools used for development, the ways devices communicate with each other, and the extra parts of the system that support the main functions.

CHAPTER 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

Following the detailed introduction of the project in Chapter 1, this chapter presents the technical environment and the main components used in the development of the IOBOX platform. It introduces the hardware and software resources that form the basis of the system, including the STM32G474 microcontroller, external peripherals, sensors, development tools, and communication protocols.

2.2 General Concept of the IOBOX Platform

As indicated in the previous chapter, the project's goal is to develop a configurable industrial input/output system designed to facilitate the integration of sensors, actuators, and industrial communication interfaces within IoT and automation systems.

The platform serves as an intermediary between field devices and supervisory systems by acquiring data from sensors, processing the collected information, controlling external devices and communicating using various communication protocols.

The use of a modular architecture allowed the integration of different types of hardware modules and software functionalities according to application needs. This flexibility makes the IOBOX suitable for a wide range of industrial monitoring and data acquisition applications.

2.3 IOBOX Hardware architecture

The IOBOX platform provides a set of analog, digital, and mixed-signal I/O resources. It includes four analog input channels connected to the ADC peripherals and seven digital inputs. On the output side, the device offers seven digital outputs, four PWM channels for actuator control, and DAC-based outputs capable of generating industrial 4–20 mA current signals. This enables direct interaction with industrial control and monitoring systems.

The platform supports multiple communication interfaces, ensuring compatibility with industrial and IoT environments. These include USB for data exchange, SPI for high-speed communication, and I^2C buses for sensor integration and peripheral expansion. Industrial connectivity is implemented through dedicated transceivers: TCAN330GD for CAN FD, TJA1020 for LIN, and THVD1410 for RS485, enabling robust long-distance communication and Modbus RTU support.

To enhance communication flexibility, the architecture integrates a TCA9548A I^2C multiplexer, expanding a single I^2C bus into eight independent channels. In addition, a PCF8574 I/O expander drives CD74HC4051 multiplexers, allowing selection among multiple UART channels while minimizing the number of required microcontroller pins.

All interfaces are routed to external connectors, simplifying integration with sensors, actuators, and industrial equipment. This modular architecture ensures scalability for automation, monitoring, data acquisition, and IoT applications, while allowing future hardware and software extensions.

As illustrated in Figure 2.1, the overall architecture of the IOBOX platform is centered on the STM32G474MBTx microcontroller, which manages data acquisition, processing, communication, and control tasks.

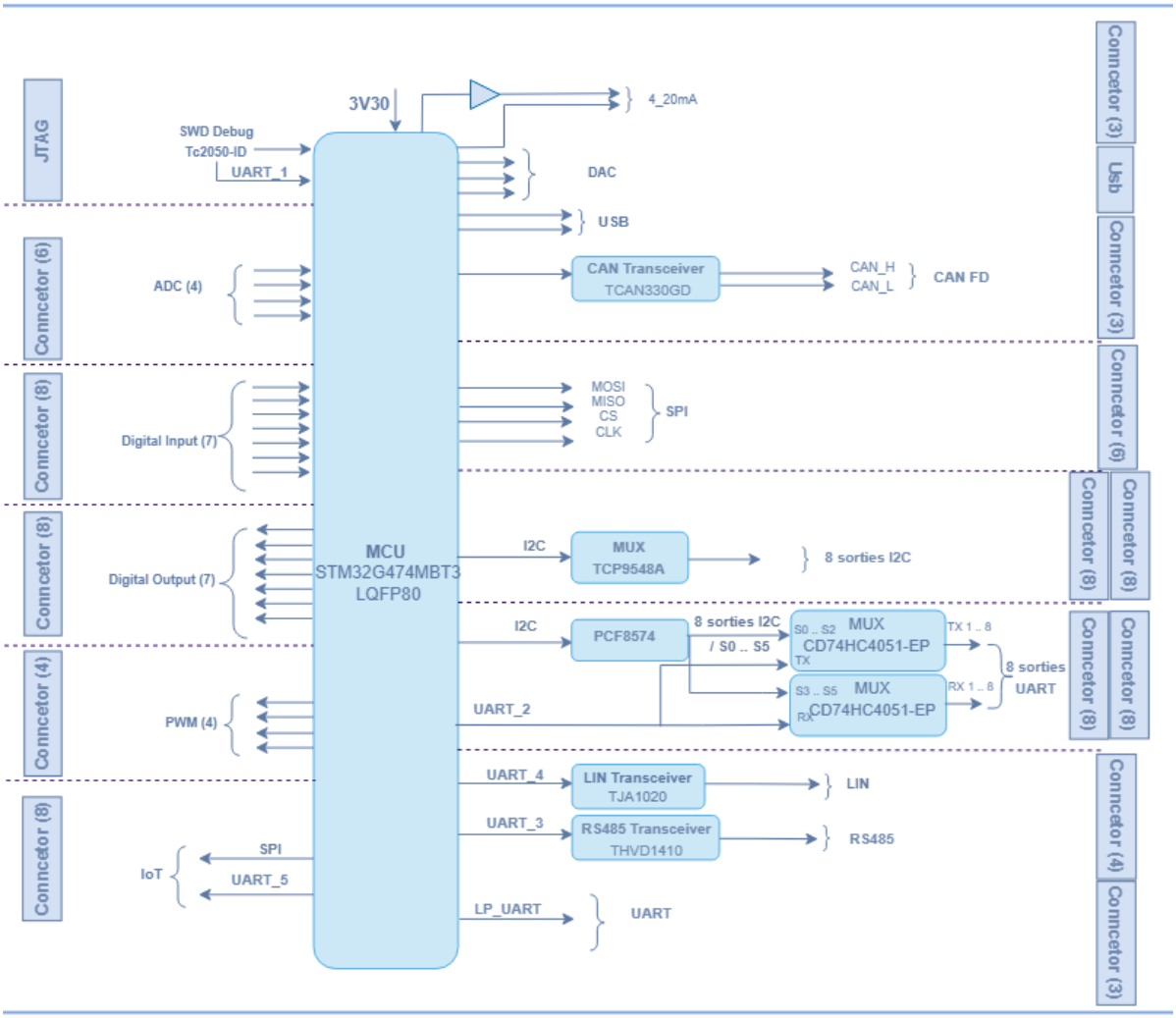

Figure 2.1: Hardware architecture of the IOBOX platform

2.4 Hardware components

2.4.1 STM32G474MBTx Microcontroller

At the heart of the IO-Box hardware architecture lies the STM32G474 microcontroller, a high-performance device from STMicroelectronics. It is based on the ARM Cortex-M4 core with floating-point unit (FPU), making it particularly well-suited for real-time signal processing and control applications.

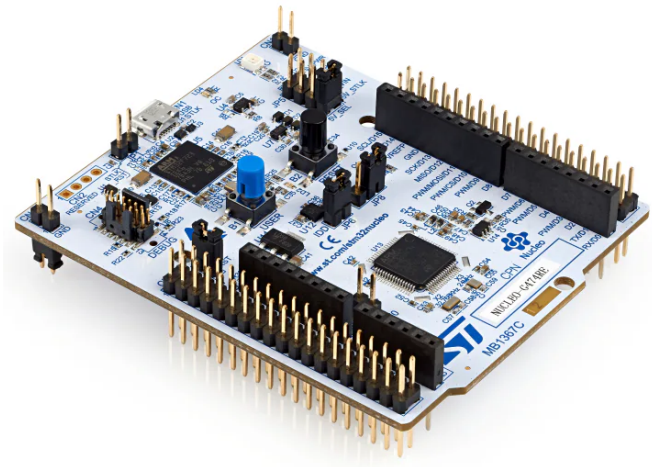

Figure 2.2: STM32G474 Microcontroller (physical package)

The STM32G474 was preferred over the STM32F4 series due to its higher ADC count (five independent ADC units versus two on the F4), its native FDCAN peripheral, and its more modern HAL ecosystem. Compared to the STM32L4 series, the G474 offers higher clock speed (170 MHz versus 80 MHz) and a richer analog front end, which are necessary for real-time signal processing at the required sampling rates.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26 × 12-bit; DAC: 7 × 12-bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	–40 °C to +125 °C
Mounting Type	Surface Mount
Package	64-LQFP (10 × 10 mm)
Base Product Number	STM32G474

Within the IOBOX, the STM32G474 acts as the central processing unit, coordinating signal acquisition, local data processing, and actuator control. It also supervises communication with external systems through multiple industrial protocols, making it a key enabler of the project's objectives.

2.4.2 EEPROM Memory: M95M01-RDW6TP

To complement the STM32 microcontroller and ensure reliable data storage, the IOBOX integrates an external EEPROM memory, specifically the M95M01-RDW6TP from STMicroelectronics. This device provides non-volatile storage for configuration parameters, calibration data, and system logs, allowing the platform to retain critical information even after power cycles.

The M95M01-RDW6TP is a 1-Mbit serial EEPROM organized as 131,072 × 8 bits. It features a fast access time of 25 ns and communicates via the SPI bus, ensuring efficient integration with the STM32G474. Packaged in an 8-pin SOIC format, it offers a compact solution suitable for embedded applications. Its endurance of up to one million write cycles and data retention of 40 years make it a robust choice for industrial environments where reliability is essential.

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit (131,072 × 8 bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

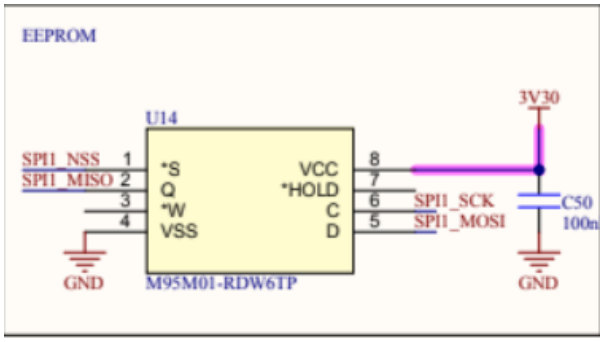

Figure 2.3: M95M01-RDW6TP EEPROM memory chip

2.4.3 Sensors

In the IOBOX project, several sensors were integrated to enable environmental and physical measurements. Our work focused not only on interfacing these sensors with the

STM32G474 microcontroller, but also on configuring their registers, managing communication protocols, and performing calibration to ensure accurate data acquisition. Each sensor was studied in detail, and its characteristics were documented to highlight its role within the system.

ENS210 Sensor

The ENS210 is a digital sensor from ScioSense that integrates both temperature and humidity measurement in a compact package. It communicates via an I^2C interface, making it easy to integrate with microcontrollers such as the STM32G474. Its high accuracy and low power consumption make it suitable for environmental monitoring in embedded systems.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	**Specification**
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I^2C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

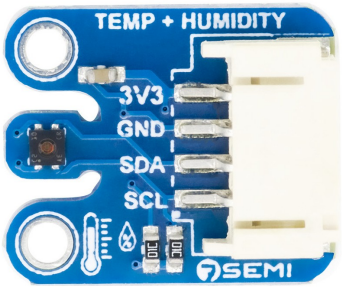

Figure 2.4: ENS210 digital temperature and humidity sensor

OPT3001DNPR Sensor

The OPT3001DNPR is a digital ambient light sensor from Texas Instruments. It is designed to measure the intensity of visible light and provide results in lux units, making it ideal for applications such as automatic brightness control, display backlight adjustment, and environmental monitoring. The sensor communicates via the I^2C protocol and features built-in calibration and linear response across a wide dynamic range.

In the IO-Box project, the OPT3001DNPR was integrated to provide accurate light measurements. Work was carried out on its internal registers to configure measurement modes, thresholds, and calibration routines, ensuring reliable data acquisition for real-time applications.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	**Specification**
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I^2C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	–40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)

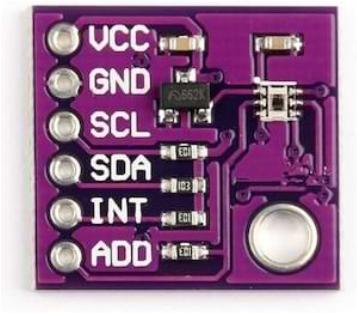

Figure 2.5: OPT3001DNPR ambient light sensor

HTS221TR Sensor

The HTS221TR is a capacitive digital humidity and temperature sensor developed by STMicroelectronics. It integrates sensing elements and signal conditioning circuitry within a compact package, providing calibrated measurements through a standard I^2C interface. The sensor is designed for low-power applications while maintaining good measurement accuracy, making it suitable for environmental monitoring and industrial IoT systems.

Within the IOBOX platform, the HTS221TR is used to acquire ambient temperature and relative humidity data. Its factory-calibrated coefficients simplify integration and improve measurement reliability. Communication with the STM32G474 microcontroller is achieved through the I^2C bus, and dedicated software drivers were developed to configure the device and process sensor data.

Table 2.5: Key Characteristics of HTS221TR Sensor

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Temperature + Humidity Sensor
Interface	I^2C (SPI mode not used in this project)
Supply Voltage	1.7 V – 3.6 V
Temperature Range	-40 °C to +120 °C
Temperature Accuracy	±0.5 °C (typ.)
Humidity Range	0 – 100 % RH
Humidity Accuracy	±3.5 % RH (typ.)
Output Resolution	16-bit
Package	HLGA-6 (2 × 2 mm)

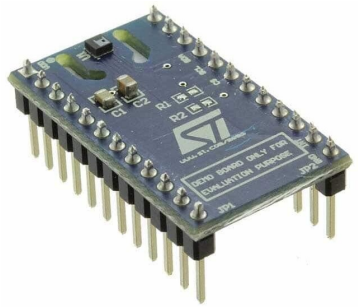

Figure 2.6: HTS221TR temperature and humidity sensor

2.5 Peripherals

2.5.1 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter (DAC) is an analog output peripheral integrated within the STM32G474MBTx microcontroller. It converts digital values stored in memory into continuous analog voltage levels with a resolution defined by the DAC bit width. This allows the generation of precise voltage references or analog waveforms such as sine waves, ramps, or arbitrary signals.

The DAC can operate in several modes, including software-triggered mode, timer-triggered mode, and DMA-driven mode. In timer-triggered mode, a hardware timer periodically updates the DAC output, ensuring precise and deterministic signal timing. When combined with Direct Memory Access (DMA), the DAC can continuously output precomputed waveform data without CPU intervention, significantly improving efficiency.

In the IOBox platform, the DAC is used to generate analog reference signals for actuator control. Two DAC channels are available, supporting both software-triggered and timer-triggered output modes.

An example of DAC signal generation is illustrated in Figure 2.7.

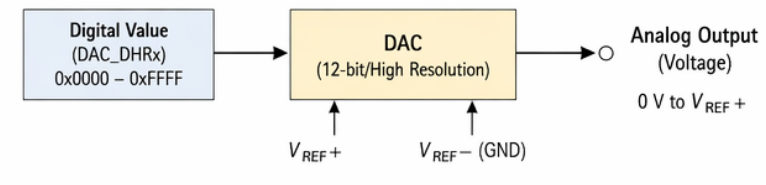

Figure 2.7: Example of analog signal generated by DAC

2.5.2 Analog to Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) is a fundamental peripheral in embedded systems, responsible for transforming continuous analog signals into discrete digital values that can be processed by the microcontroller. This conversion is essential for applications where physical phenomena such as temperature, pressure, or current must be measured and interpreted by digital logic.

In the IOBox project, ADCs are employed to acquire signals from sensors and external modules, enabling precise monitoring and control. The STM32G474 microcontroller integrates multiple high-resolution ADC channels, allowing simultaneous sampling of different inputs. This capability is particularly important in industrial contexts where multiple signals must be captured in real time.

The operation of an ADC follows a well-defined process. The analog input is sampled at regular intervals determined by the sampling frequency. Each sample is then quantized into a digital value according to the converter's resolution (e.g., 12-bit or 16-bit). The resulting digital word represents the amplitude of the analog signal at that instant. The accuracy of this conversion depends on factors such as resolution, sampling rate, and reference voltage stability.

Key characteristics of ADCs include:

- **Resolution**: Defines the number of discrete levels available.
- **Sampling Rate**: Determines how frequently the analog signal is measured, affecting the fidelity of dynamic signals.
- **Reference Voltage**: Sets the maximum measurable input range, ensuring proper scaling of the digital output.
- **Input Channels**: Multiple channels allow acquisition from several sensors simultaneously.
- **Conversion Modes**: Support for single, continuous, and DMA-driven conversions, enabling flexible integration into firmware.

Within the IO-Box, ADCs are used to capture sensor data such as voltage levels, current measurements, or other analog signals. These values are then processed by the firmware to generate meaningful information, trigger control actions, or communicate results to external supervisory systems.

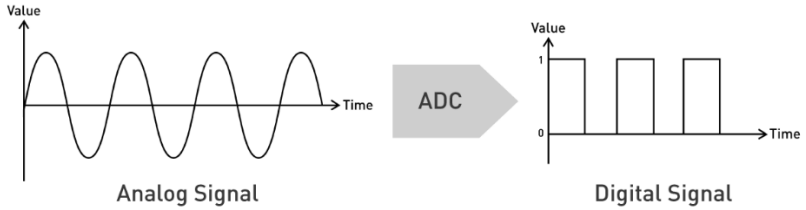

Figure 2.8: Basic ADC conversion from analog input to digital output

2.5.3 Timers

Timers are fundamental peripherals in the STM32G474 microcontroller, designed to provide accurate time base generation, event counting, and signal measurement. They operate by incrementing or decrementing a counter register according to a clock source, and can trigger actions when predefined values are reached.

Key functionalities of timers include:

- **Time Base Generation**: Creating precise delays or periodic events.
- **Input Capture**: Measuring external signal characteristics such as frequency or pulse width.
- **Output Compare**: Generating events when the counter matches a reference value.
- **Synchronization**: Coordinating multiple timers for complex control tasks.
- **Advanced Features**: Dead-time insertion, break inputs, and complementary outputs for safe power electronics control.

Timers are essential for scheduling tasks, measuring sensor signals, and driving peripherals that require precise timing.

2.5.4 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to encode analog information into a digital waveform by varying the duty cycle of a periodic signal. In embedded systems, PWM is widely used for motor control, LED dimming, and power regulation.

PWM signals in the STM32G474 are generated using timers. The timer counts up to a predefined value stored in the auto-reload register (ARR), while the compare register (CCR) defines the switching point of the output. The ratio between the high

time and the total period represents the duty cycle. By adjusting ARR and CCR, both frequency and duty cycle can be controlled with high precision.

Key characteristics of PWM include:

- **Frequency**: Determined by prescaler and ARR configuration.
- **Duty Cycle**: Defined by CCR relative to ARR.
- **Resolution**: Limited by timer bit width (typically 16-bit).
- **Complementary Outputs**: Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications**: Motor drivers, LED dimming, switching converters, and DAC emulation.

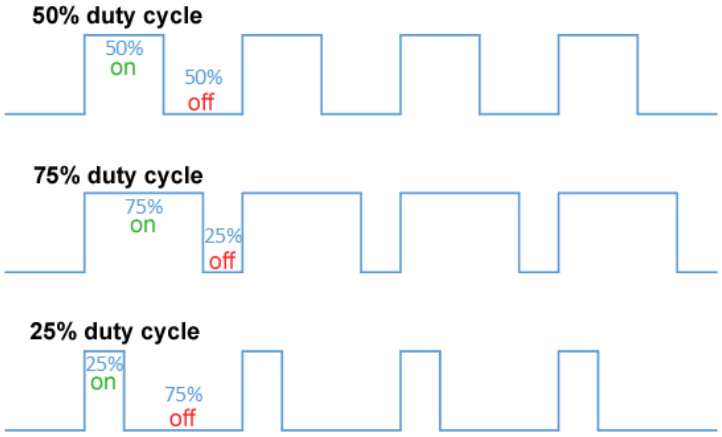

Figure 2.9: Illustration of PWM signals with different duty cycles

2.6 Communication Protocols

2.6.1 I2C Protocol

Inter-Integrated Circuit (I2C) is a synchronous, half-duplex serial communication protocol widely used for communication between integrated circuits over short distances. It relies on two bidirectional lines: the Serial Data Line (SDA) and the Serial Clock Line (SCL), both requiring pull-up resistors. The protocol follows a master-slave architecture, where the master initiates communication and generates the clock signal.

Each device connected to the bus is identified by a unique address. Data is then exchanged one byte at a time, with each byte followed by an acknowledgment bit (ACK/NACK) from the receiving device. The communication is terminated by a

STOP condition. I2C supports different speed modes, including Standard Mode (100 kHz), Fast Mode (400 kHz), and higher-speed variants.

The I2C frame structure is illustrated in Figure 2.10.

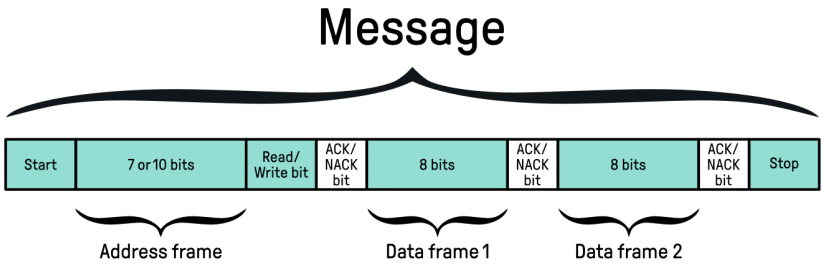

Figure 2.10: I2C frame format

In this project, the I2C interface of the STM32G474MBTx is configured using the HAL library generated by STM32CubeMX. It is used to communicate with external peripherals such as sensors and low-speed devices. The implementation relies on interrupt or polling-based data transfers depending on timing constraints. Particular attention is given to bus stability, including proper pull-up resistor sizing and handling of potential communication errors such as bus busy states or missing acknowledgments.

As shown in Figure 2.11, the I2C bus allows multiple slave devices.

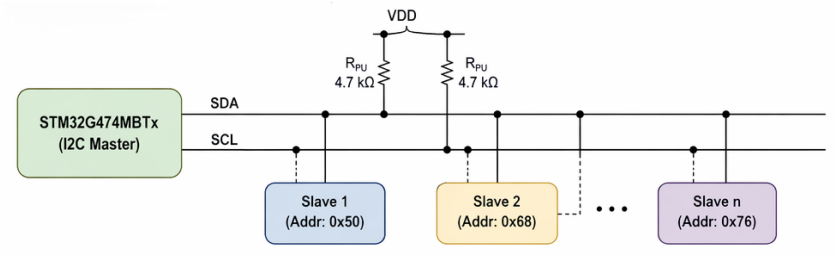

Figure 2.11: I2C communication bus architecture

2.6.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master–slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IOBOX project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed

make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. This process is synchronized by the clock (SCLK), ensuring deterministic timing. The communication sequence typically follows these steps: the master activates the Chip Select (CS) line of the desired slave, provides the clock signal, transmits data bit-by-bit on MOSI while receiving data on MISO, samples the data according to the configured mode, and finally deactivates the CS line to end the session. This mechanism allows full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI supports four modes of operation (Mode 0 to Mode 3), determined by the configuration of clock polarity (CPOL) and clock phase (CPHA). This flexibility ensures compatibility with a wide range of peripheral devices. Key characteristics of SPI include its high speed (operating at several MHz), simple wiring with four main signals (MOSI, MISO, SCLK, CS), and scalability through multiple slave connections.

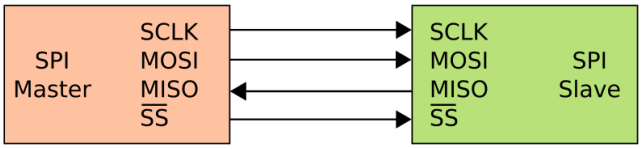

Figure 2.12: Basic SPI communication between master and slave devices

2.6.3 UART Protocol

Universal Asynchronous Receiver-Transmitter (UART) is an asynchronous serial communication protocol that enables full-duplex data exchange between devices without a shared clock signal. Data is transmitted over two lines: Transmit (TX) and Receive (RX). Synchronization between devices is achieved by configuring identical communication parameters, including baud rate, data bits, parity, and stop bits.

Each data frame typically consists of a start bit, followed by a configurable number of data bits (usually 8), an optional parity bit for error detection, and one or more stop bits. Unlike synchronous protocols, UART does not include a clock line, making it simpler but more sensitive to timing mismatches.

In this project, UART is used for communication between the STM32 microcon-

troller and external systems such as a host computer or other embedded modules. It plays a key role in debugging, logging, and real-time data exchange. The implementation uses the STM32 HAL drivers with support for interrupt-driven and DMA-based transmission to ensure efficient and non-blocking communication. Error handling mechanisms, including overrun, framing, and noise errors, are also considered to improve communication reliability.

The UART frame structure is illustrated in Figure 2.13.

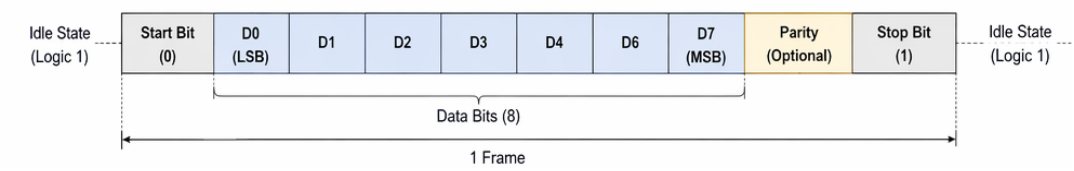

Figure 2.13: UART frame format

2.6.4 RS485 Communication

RS485 is a robust differential serial communication standard widely used in industrial environments due to its high noise immunity and long-distance capabilities. Unlike single-ended UART communication, RS485 transmits data over a differential pair of lines (A and B), where the information is encoded as a voltage difference between the two conductors. This approach significantly reduces susceptibility to electromagnetic interference and ground potential differences.

The RS485 standard supports half-duplex communication in multi-drop configurations, allowing multiple nodes to share the same bus. However, only one device can transmit at a time, which requires proper bus arbitration at the software level or through protocol design.

In this project, RS485 serves as the physical layer for Modbus RTU communication. A dedicated UART peripheral (USART3) is used with an external RS485 transceiver to convert TTL logic levels to differential signals. Direction control is managed in software, as described in Chapter 3.

Before transmission, the microcontroller sets the DE pin high to enable the driver stage. After sending the data frame, the DE pin is reset to return the transceiver to reception mode. This timing control is critical to avoid bus contention. The communication is typically used for reliable data exchange in noisy environments, especially when longer cable lengths are required compared to standard UART.

Figure 2.14 illustrates a typical RS485 bus configuration.

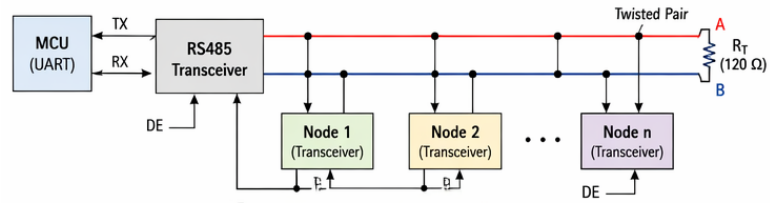

Figure 2.14: RS485 differential bus with multi-node configuration

2.6.5 Modbus Protocol

Modbus is one of the most widely used industrial communication protocols for exchanging data between electronic devices. Originally developed by Modicon in 1979, it follows a client-server (master-slave) architecture and enables communication between programmable logic controllers (PLCs), sensors, actuators, Human-Machine Interfaces (HMIs), and embedded systems.

Modbus defines a standardized message structure that allows devices from different manufacturers to communicate using a common protocol. Two main variants are commonly used: Modbus RTU, which operates over serial communication interfaces such as RS232 and RS485, and Modbus TCP, which operates over Ethernet networks.

In the IOBOX project, Modbus RTU is implemented over the RS485 physical layer to ensure reliable communication in industrial environments. The STM32G474 microcontroller acts as either a Modbus master or slave depending on the application requirements. Through Modbus registers, external systems can access sensor measurements, configuration parameters, status information, and control commands.

In this project, the IOBox operates as a Modbus RTU slave at address 0x01, communicating over RS485 at 19,200 bps. The slave implementation supports function codes FC01 through FC06, FC15, and FC16. Input registers expose real-time sensor measurements (temperature, humidity, illuminance) to any connected Modbus master, and holding registers allow actuator control. The implementation details are described in Chapter 3.

A Modbus RTU frame consists of four main fields:

- **Slave Address**: Identifies the target device.
- **Function Code**: Specifies the requested operation.
- **Data Field**: Contains transmitted data or register information.
- **CRC Field**: Provides error detection for frame integrity.

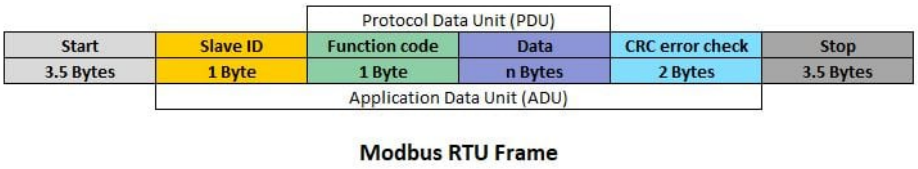

Figure 2.15: Typical Modbus RTU frame structure

To ensure compatibility with standard industrial devices, the implementation supports a subset of Modbus function codes, classified according to their role in data access and control:

Table 2.6: Supported Modbus Function Codes in the IOBOX Platform

Function Code	**Name**	**Description**
FC01	Read Coils	Reads digital output states (ON/OFF status)
FC02	Read Discrete Inputs	Reads digital input signals from sensors or switches
FC03	Read Holding Registers	Reads configuration and control registers (16-bit)
FC04	Read Input Registers	Reads real-time measurement data (sensor values)
FC05	Write Single Coil	Controls a single digital output (ON/OFF command)
FC06	Write Single Register	Writes a single 16-bit configuration or control value
FC15	Write Multiple Coils	Updates multiple digital outputs in one request
FC16	Write Multiple Registers	Writes multiple registers for bulk configuration/control

This classification enables structured interaction with the IOBOX memory map, separating real-time acquisition data (input registers) from controllable parameters (holding registers). It also ensures interoperability with a wide range of Modbus-compatible supervisory systems such as SCADA platforms and PLCs.

2.6.6 Controller Area Network (CAN)

The Controller Area Network (CAN) is a robust serial communication protocol originally developed for automotive applications, but now widely used in industrial and embedded systems. It enables reliable data exchange between multiple devices without requiring a central host, making it ideal for distributed control systems.

In the IOBOX project, CAN is employed to ensure communication with industrial equipment and sensors that demand high reliability and fault tolerance. Its ability to handle multiple nodes on a single bus makes it particularly suitable for environments where numerous devices must share information in real time.

CAN communication is based on a message-oriented protocol. Instead of addressing specific devices directly, each message carries an identifier that defines its priority and content. This allows multiple nodes to listen simultaneously and decide whether to act on the received data. The arbitration mechanism ensures that when two nodes attempt to transmit at the same time, the message with the highest priority (lowest identifier value) wins, while the other node waits until the bus is free. This guarantees deterministic communication without collisions.

Key characteristics of CAN include:

- **Multi-master capability**: Any node can initiate communication when the bus is idle.
- **Error detection and handling**: Built-in mechanisms such as CRC checks, acknowledgment bits, and error frames ensure data integrity.
- **Priority-based arbitration**: Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability**: Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed**: Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

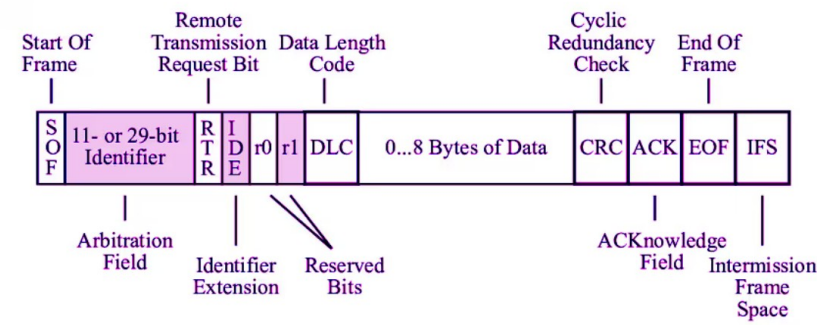

Figure 2.16: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

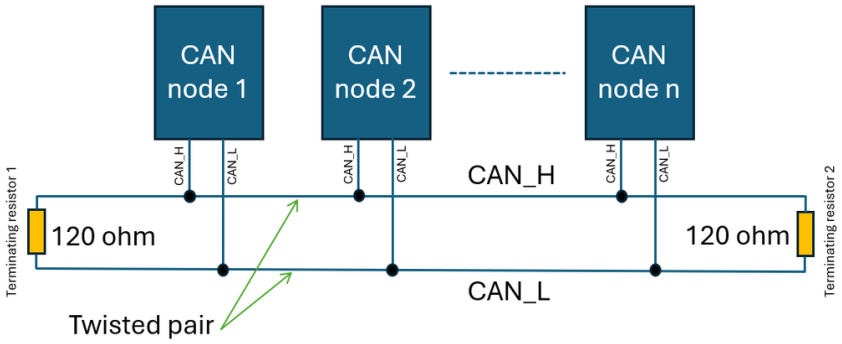

Figure 2.17: Basic CAN bus communication with multiple nodes

2.6.7 LIN Protocol

Local Interconnect Network (LIN) is a low-cost serial communication protocol designed for distributed control systems, particularly in automotive applications. It operates on a single-wire physical layer and follows a strict master-slave architecture where the master node controls the entire bus communication schedule.

Unlike event-driven protocols, LIN is time-triggered, meaning communication is organized into predefined frames sent at fixed intervals. Each frame consists of a header transmitted by the master and a response transmitted by a slave node. The header includes a break field (used to signal the start of a frame), a synchronization byte, and an identifier field that determines which node should respond.

The data response typically contains up to 8 bytes, followed by a checksum used for error detection. LIN achieves deterministic timing by enforcing strict scheduling, making it suitable for non-critical but predictable control tasks.

In this project, LIN communication is implemented using the STM32 UART peripheral configured in LIN mode. The hardware support simplifies break detection and synchronization field handling, reducing software complexity. The system can therefore reliably detect frame boundaries and process incoming data with minimal CPU overhead.

The LIN frame structure is shown in Figure 2.18.

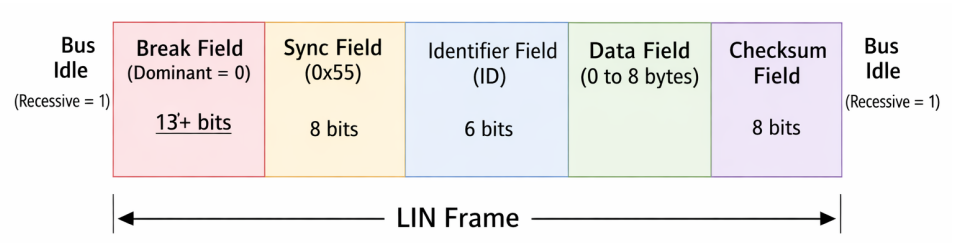

Figure 2.18: LIN frame structure

2.7 Software tools

2.7.1 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor optimized for modern software development. It supports multiple programming languages and provides advanced features such as syntax highlighting, intelligent code completion, and debugging tools[3].

In this project, Visual Studio Code was used as the primary text editor for documentation, script editing, and Git-based version control. Extensions such as Git Graph enabled visualization of commit history, branch management, and tracking of code evolution across the development team.

Figure 2.19: Visual Studio Code interface

2.7.2 IAR Embedded Workbench

IAR Embedded Workbench (IAR EW) is a professional integrated development environment (IDE) dedicated to embedded systems programming, particularly for microcontrollers such as the STM32 family. It provides a highly optimized C/C++ compiler, advanced debugging capabilities, and efficient project management tools[4].

In this project, IAR EW was used for firmware development, compilation, and debugging. Its powerful optimization features contributed to generating efficient and reliable embedded code, which is critical for real-time applications and resource-constrained systems.

Figure 2.20: IAR Embedded Workbench IDE

2.7.3 STM32CubeMX

STM32CubeMX is a graphical configuration tool developed by STMicroelectronics for STM32 microcontrollers. It allows developers to easily configure peripherals, clock settings, and pin assignments through an intuitive interface. The tool also generates initialization code based on the selected configuration, significantly reducing development time and minimizing configuration errors[5].

In this project, STM32CubeMX was used in a limited capacity: it was invoked once at the beginning of the project to generate the initial project skeleton, configure the system clock (HSI PLL at 170 MHz), and enable the Serial Wire Debug interface. The peripheral initialization functions that CubeMX would normally generate were intentionally not used. Instead, all peripheral drivers were developed as handwritten BSP (Board Support Package) modules, which are described in detail in Chapter 3. CubeMX thus served exclusively as a project scaffolding and clock configuration tool.

Figure 2.21: STM32CubeMX configuration interface

2.7.4 GitLab

GitLab was used as the version control and collaboration platform throughout the project. The repository was organized by module (BSP, Managers, Application), with branches used to isolate feature development and avoid conflicts during parallel work. Commit history served as a development log, and merge requests were used to review code before integrating changes into the main branch.

Figure 2.22: GitLab logo

2.7.5 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be

dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

CHAPTER 3

DESIGN AND PRACTICAL IMPLEMENTATION

3.1 Introduction

In this chapter, we present the design and practical implementation of the IOBOX platform. Building on the requirements and hardware foundations introduced in the previous chapters, we describe the key design choices and software architecture that transform these elements into a functional embedded system. The discussion provides a concise overview of the layered implementation approach and the main technical decisions that ensure modularity, scalability, and suitability for industrial applications.

3.2 Software Architecture

The firmware of the IOBOX platform follows a structured four-layer architecture. At the lowest level, the hardware abstraction layer (HAL) that provides direct communication with the hardware. Above it, the Board Support Package (BSP) handles board-specific configuration including bus initialization and public API. The third layer, a Manager layer encapsulating peripheral control and communication protocol logic, as a higher-level API. At the top, an application layer that implements the system's business logic and task management and execution. This strict architecture enforces a clear separation of the code layers ensuring that each layer can only interact with the APIs of the layer beneath it. A central configuration file, `feature_config.h`, governs the compile-time inclusion of every hardware feature through preprocessor flags, ensuring that only the peripherals and protocols required for a given deployment are compiled into the firmware binary.

A single global structure, `SystemBoard`, declared in `feature_config.h` and instantiated in `main.c`, serves as the shared data exchange point between all layers. It aggregates real-time sensor readings, CAN frame data, and status information, and is accessed directly by the Manager and Application layers during each execution cycle.

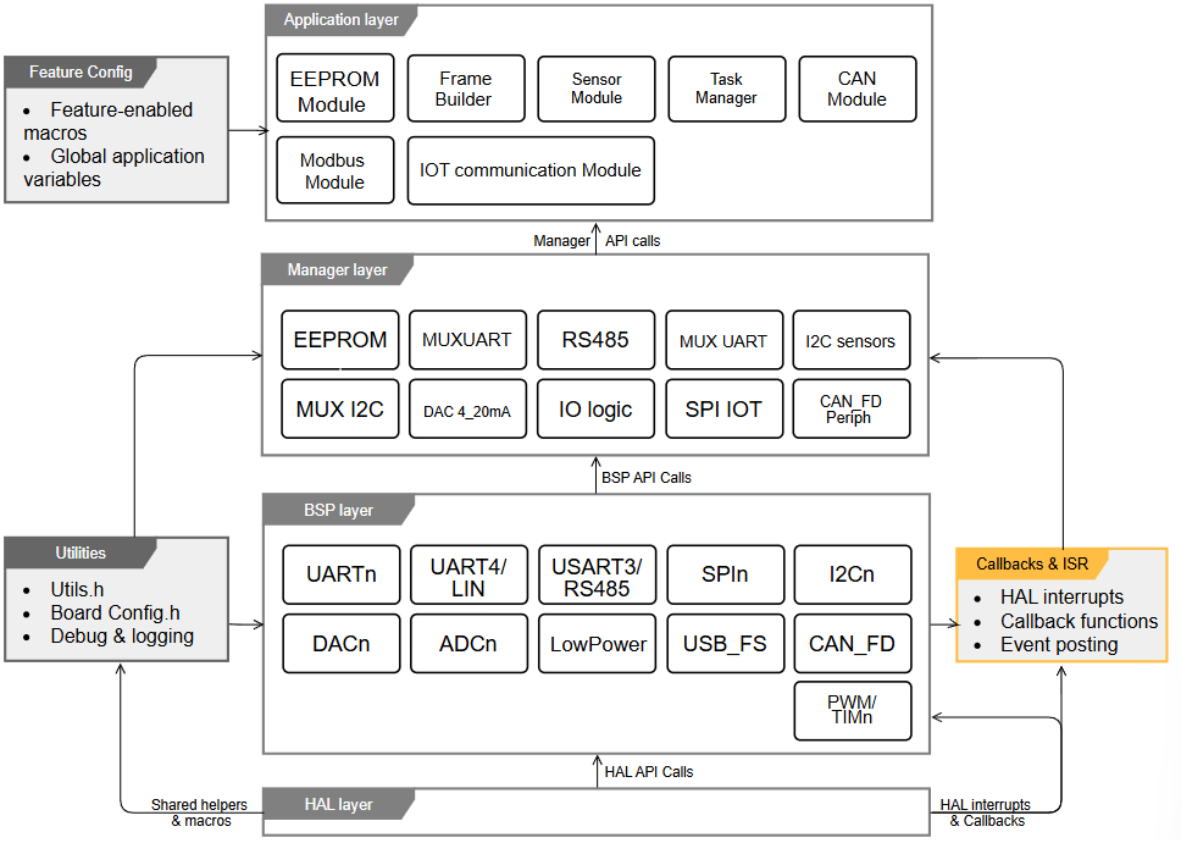

Figure 3.1: Layered software architecture of the IOBOX platform

The following subsections will describe each layer in detail, from the lowest software layer that interfaces directly with the hardware to the highest layer that implements application logic.

3.2.1 Board Support Package Layer

The Board Support Package constitutes the lowest layer of the firmware stack. It provides a collection of peripheral driver modules, each encapsulating the initialisation, configuration, and data transfer routines for a single hardware interface. By confining all register-level and HAL-level operations to this layer, the upper layers are shielded

from hardware-specific details and can interact with peripherals exclusively through the BSP's public API.

Each BSP module follows a consistent structure: a header file located under `BSP/Inc/` declares the public API, while the corresponding source file under `BSP/Src/` contains the implementation. The platform exposes BSP modules for the following peripheral categories.

Serial communication interfaces are covered by `BSP_USART1`, `BSP_USART2`, `BSP_UART5`, and `BSP_LPUART1` for general-purpose asynchronous communication, by `BSP_RS485` for the half-duplex differential bus used by the Modbus RTU stack, by `BSP_LIN` for the single-wire LIN bus, and by `BSP_UART_MUX` for the software-controlled UART channel multiplexer. High-speed synchronous communication is handled by three independent SPI drivers: `BSP_SPI1`, `BSP_SPI2`, and `BSP_SPI4`. Sensor and peripheral expansion buses are served by `BSP_I2C2` and `BSP_I2C3`, complemented by `BSP_I2C_MUX`, which controls the TCA9548A eight-channel I^2C multiplexer. The CAN interface is abstracted by `CAN1_BSP`, and digital input/output operations are centralised in `BSP_DigitalIO`.

Analog peripherals are equally covered: four ADC drivers (`ADC1_BSP`, `ADC3_BSP`, `ADC4_BSP`, `ADC5_BSP`) expose acquisition functions for each active conversion channel, while two DAC drivers (`BSP_DAC1` and `BSP_DAC2`) provide voltage output generation. Four timer drivers (`TIM1_BSP` through `TIM4_BSP`) manage time-base and PWM generation. A low-power management module, `BSP_LowPower`, groups the power mode transition routines.

This modular decomposition means that porting the firmware to a different STM32 variant requires changes only within the BSP layer, leaving the Manager and Application layers unmodified.

3.2.2 Manager Layer

Three sensor manager modules were developed, one per physical sensor, each handling I^2C multiplexer channel selection, register-level communication, and raw-to-physical conversion.

The HTS221TR manager drives the STMicroelectronics capacitive humidity and temperature sensor connected to I^2C multiplexer channel 0. Subsequent acquisitions apply linear interpolation between these reference points to convert the 16-bit raw ADC output into calibrated temperature in degrees Celsius and relative humidity in percent, as shown in Equations 3.1 and 3.2.

$$\%\mathrm{RH} = H_{0,\mathrm{rH}} + \frac{(H_{1,\mathrm{rH}} - H_{0,\mathrm{rH}}) \cdot (\mathrm{raw} - H_{0,T_0})}{H_{1,T_0} - H_{0,T_0}} \tag{3.1}$$

$$T\,[^\circ\mathrm{C}] = T_{0,\mathrm{degC}} + \frac{(T_{1,\mathrm{degC}} - T_{0,\mathrm{degC}}) \cdot (\mathrm{raw} - T_{0,\mathrm{out}})}{T_{1,\mathrm{out}} - T_{0,\mathrm{out}}} \tag{3.2}$$

Data readiness is verified by polling the sensor's status register before each acquisition. If the data-ready flag is not asserted within a configurable retry count, the acquisition routine returns a timeout status without blocking the system further.

The ENS210 manager interfaces with the ScioSense combined temperature and humidity sensor on multiplexer channel 2. The module operates in single-shot mode: a measurement is triggered by writing to the sensor's start register, a fixed 10-millisecond settling delay is applied, and the 16-bit raw temperature and humidity registers are then read. Conversion follows the datasheet-specified formulae: temperature is obtained as $T_K = \mathrm{raw}/64$, then converted to Celsius by subtracting 273.15; relative humidity is obtained as $\%\mathrm{RH} = \mathrm{raw}/512$.

The OPT3001DNPR manager drives the Texas Instruments ambient light sensor on multiplexer channel 1. The sensor is configured at initialisation by writing a control word to its configuration register, selecting continuous conversion mode, automatic full-scale range, and an 800-millisecond conversion time. The 16-bit result register encodes the measurement as a four-bit binary exponent and a twelve-bit mantissa; the illuminance in lux is recovered by the expression $E_{\mathrm{lux}} = \mathrm{mantissa} \times 0.01 \times 2^{\mathrm{exponent}}$.

In all three managers, the $\mathrm{I}^2\mathrm{C}$ multiplexer channel is selected at the start of each transaction and disabled upon completion, preventing bus contention between sensors sharing the same physical $\mathrm{I}^2\mathrm{C}$ bus.

Modbus RTU Manager

The Modbus RTU stack provides both master and slave roles over the RS485 physical layer and is entirely self-contained within a single manager module. The slave role, which is active in the current platform configuration, operates through interrupt-driven reception. Eight function codes are supported: FC01, FC02, FC03, FC04, FC05, FC06, FC15, and FC16. Frames addressed to other slaves are silently discarded; frames with invalid CRC are dropped without response, as required by the Modbus RTU specification. Following each handled frame, the receiver is re-armed immediately to avoid missing subsequent requests.

CAN Manager

CAN frame reception is managed by a dedicated module that decouples the asynchronous nature of CAN bus traffic from the periodic execution rhythm of the Application layer. Because CAN frames can arrive at any time, independently of the firmware's scheduling cycle, the manager introduces an intermediate buffering stage between the BSP reception interface and the rest of the system. Incoming frames are captured and held as they arrive; the Application layer then retrieves and consumes them at its own cadence without any risk of frame loss during burst traffic conditions. Once processed, the received data is written into the CAN-related fields of the shared `SystemBoard` structure, making it immediately available to any other module that consults the platform's central data store.

EEPROM Manager

Non-volatile data persistence is managed by `EEPROM_manager.c`, which interfaces with the M95M01-RDW6TP serial EEPROM via the SPI BSP layer. The manager exposes store and load primitives consumed by the Application layer to persist the `DataFrame` structure across power cycles.

3.2.3 Filter Manager

Industrial sensor signals are rarely clean: electrical interference, thermal gradients, and inherent sensor noise introduce unwanted fluctuations into raw measurements that, if left unprocessed, degrade the accuracy of the data delivered to the application layer. To address this requirement without coupling noise-reduction logic to individual sensor drivers, the IOBOX firmware provides a dedicated, generic signal processing module implemented in `Filter_Manager.c` and `Filter_Manager.h`.

The six filter algorithms provided by the module are described individually below.

Exponential Moving Average Filter

Purpose. The Exponential Moving Average (EMA) is a low-pass filter designed to smooth noisy sensor readings by weighting recent samples more heavily than older ones while discarding neither.

Principle. Each output value is a weighted combination of the current raw input and the previous filtered output. The weight assigned to the new sample is controlled by a single configurable parameter, so the filter can be made to respond quickly to real signal changes or to suppress short-lived noise spikes, depending on the application.

Main Equation.

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1] \tag{3.3}$$

Parameters. $x[n]$ is the current raw sample, $y[n-1]$ is the previous filtered output stored in `prev_output`, and $\alpha \in (0, 1]$ is the smoothing factor: values near zero produce heavy smoothing with slow response, while values near one allow fast tracking with minimal noise reduction.

Advantages. The EMA requires only two floating-point multiplications and one addition per update step, making it the least computationally demanding filter in the library. Its constant memory footprint and the absence of a ring buffer make it directly suitable for resource-constrained microcontrollers. The single parameter `alpha` provides a simple and intuitive tuning knob.

Disadvantages. Because all past samples influence the output with exponentially decaying weights, the EMA has no finite settling time after a large step change in the input: the output approaches the new level asymptotically. Additionally, the degree of smoothing depends implicitly on the sampling rate; if the rate changes, the effective cutoff behaviour changes as well unless `alpha` is retuned.

Use within the IOBOX. The EMA is the recommended starting filter for continuous sensor channels such as the temperature readings from the HTS221TR and ENS210 sensors, where measurements arrive at a fixed 1000 ms period and the primary concern is reducing high-frequency noise without introducing noticeable lag.

Simple Moving Average Filter

Purpose. The Simple Moving Average (SMA) is a low-pass filter that smooths a signal by computing the arithmetic mean of a fixed number of the most recent samples. It is used when a consistent, equal-weighted average over a sliding observation window is preferred over recursive weighting.

Principle. A fixed-size window of recent samples is maintained in a circular ring buffer. As each new sample arrives, it replaces the oldest value in the buffer and the average is recomputed. A running sum is maintained so that the average requires only a subtraction, an addition, and a division per update, rather than recomputing the full sum from scratch.

Main Equation.

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{3.4}$$

Parameters. N is the window size, configurable between 1 and `FILTER_SMA_MAX_WINDOW` (defined as 30 in `Filter_Manager.h`). $x[n-k]$ represents the sample collected k steps before the current one.

Advantages. Each sample contributes equally to the output, which gives the SMA a more predictable and uniform averaging behaviour than the EMA. The constant-time update algorithm means that increasing the window size does not increase computational cost per step.

Disadvantages. Memory consumption scales linearly with window size: a window of N samples requires N floating-point values in the ring buffer, equivalent to $4N$ bytes. The maximum window of 30 samples therefore consumes up to 120 bytes per instance. The SMA also introduces a latency of $(N-1)/2$ samples before it fully reflects a step change in the input.

Use within the IOBOX. The SMA is applicable to channels where equal weighting of historical samples is required or where the averaging window has a direct physical meaning, such as computing a one-minute average of lux readings from the OPT3001DNPR sensor by setting N equal to the number of 1000 ms samples in sixty seconds.

First-Order IIR High-Pass Filter

Purpose. The first-order IIR high-pass filter removes slow-varying components and DC offsets from a signal, retaining only its dynamic, rapidly changing parts. It is used when baseline drift or long-term bias needs to be eliminated from a sensor channel.

Principle. The filter computes each output as a scaled combination of the current input-to-input difference and the previous output. A configurable parameter α controls how aggressively the low-frequency content is suppressed.

Main Equation.

$$y[n] = \alpha \cdot \big(y[n-1] + x[n] - x[n-1]\big) \tag{3.5}$$

Parameters. $x[n]$ is the current input, $x[n-1]$ is the previous input stored in `prev_input`, $y[n-1]$ is the previous output stored in `prev_output`, and $\alpha \in (0,\, 1]$ is the filter coefficient.

Advantages. The IIR high-pass filter is computationally inexpensive, requiring three additions and one multiplication per update step. Its three-variable state makes it one of the most memory-efficient options for baseline removal available in the library.

Disadvantages. The output depends on both the current and past inputs and outputs, which means the filter introduces a phase shift that varies with the rate of change of the signal. This can make it less suitable for applications that require time-accurate detection of transient events.

Use within the IOBOX. This filter is appropriate for removing slow thermal drift from temperature sensor readings when only the rate of change or transient excursions are of interest to the application, rather than absolute values.

Simple Difference High-Pass Filter

Purpose. The simple difference filter is the most basic high-pass operation available in the library. It extracts the sample-to-sample variation of a signal, effectively computing a discrete approximation of its rate of change.

Principle. Each output value is simply the difference between the current input sample and the immediately preceding one. No configurable parameters are involved, making this filter parameter-free and ready to use without any tuning.

Main Equation.

$$y[n] = x[n] - x[n-1] \tag{3.6}$$

Parameters. $x[n-1]$ is the previous raw input value, stored in the `prev_input` field of the `difference` sub-structure.

Advantages. The difference filter imposes the lowest possible computational and memory cost of all filters in the library: one subtraction and one assignment per update, with one floating-point state variable. It requires no configuration and is immediately deployable on any sensor channel.

Disadvantages. Because it amplifies high-frequency components of the signal in proportion to their frequency, the difference filter is sensitive to high-frequency noise. It is appropriate only for channels that are already relatively clean or pre-filtered, or for applications where noise spikes and true events can be distinguished by magnitude.

Use within the IOBOX. The difference filter is suited to event detection on slowly sampled sensor channels, such as detecting an abrupt change in ambient light measured by the OPT3001DNPR, where a sudden illumination change produces a clearly distinguishable output spike against a near-zero background.

SMA-Subtraction High-Pass Filter

Purpose. The SMA-subtraction filter is a high-pass filter that isolates rapid signal variations by removing the local moving average from each sample. It provides baseline removal with the same ring-buffer mechanism used by the SMA low-pass filter, yielding a complementary pair whose outputs sum exactly to the original signal.

Principle. A sliding window moving average of the input is computed at each step using the same efficient running-sum ring-buffer technique as the SMA low-pass filter. The high-pass output is then obtained by subtracting this local average from the current raw sample.

Main Equation.

$$y[n] = x[n] - \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{3.7}$$

Parameters. N is the window size, subject to the same constraint as the SMA low-pass: clamped to [1, `FILTER_SMA_MAX_WINDOW`].

Advantages. Because the high-pass output is derived by subtracting a low-pass output, the signal is decomposed without any information loss: the two components always reconstruct the original sample exactly. The window size parameter has a direct and intuitive interpretation as the time span over which the local baseline is estimated.

Disadvantages. Memory usage is identical to the SMA low-pass filter: up to 120 bytes per instance at the maximum window size. The filter also introduces the same startup transient as the SMA: during the first $N-1$ samples, the baseline estimate is computed over fewer than N values, which may cause the high-pass output to be inaccurate until the buffer is filled.

Use within the IOBOX. This filter is well suited to separating short-duration illuminance events from slowly evolving ambient light baselines on the OPT3001DNPR channel, or to detecting rapid humidity transients against a slowly drifting background on either of the two humidity sensor channels.

One-State Position Kalman Filter

Purpose. The one-state Kalman filter provides an optimal estimate of a slowly changing scalar quantity by balancing the uncertainty in its own internal model against the uncertainty in each new measurement. It adapts its smoothing behaviour automatically over time, without requiring manual tuning of a fixed coefficient.

Principle. At each step, the filter first predicts how much its confidence in the current estimate has degraded (the prediction step), then corrects the estimate using the new measurement weighted by a dynamically computed gain (the update step). The gain is large when the filter is uncertain and small when it is confident, allowing it to track real changes quickly at startup and to suppress noise efficiently at steady state.

Main Equations.

$$K = \frac{P + Q}{P + Q + R} \tag{3.8}$$

$$\hat{x} = \hat{x} + K \cdot (z - \hat{x}) \tag{3.9}$$

$$P = (1 - K)\,(P + Q) \tag{3.10}$$

Parameters. Q is the process noise variance (`process_variance`), reflecting how much the true quantity is expected to vary between samples; R is the measurement noise variance (`measurement_variance`), reflecting sensor noise power; $\hat{x}$ is the current state estimate (`state_estimate`); P is the estimation error covariance (`error_covariance`); z is the new measurement; and K is the Kalman gain computed at each step.

Advantages. The one-state Kalman filter adapts its smoothing level automatically: it applies less smoothing early in a measurement session (when uncertainty is high) and progressively more as confidence accumulates. This eliminates the need to choose a fixed smoothing coefficient and makes the filter more robust to varying signal conditions than either the EMA or SMA. Its memory footprint is fixed at four floating-point variables regardless of operating conditions.

Disadvantages. The filter assumes that the true quantity changes only due to random, zero-mean process noise (constant-position model). If the quantity exhibits a systematic trend — such as a steadily rising temperature — the filter will lag behind the true value, because the model does not account for a rate of change. The two-state velocity filter described below addresses this limitation.

Use within the IOBOX. The position Kalman filter is the most suitable algorithm for the temperature and humidity channels of the HTS221TR and ENS210 sensors under stable operating conditions, where the physical environment changes slowly and randomly. It is particularly appropriate in deployments where sensor noise characteristics are known and Q and R can be set from datasheet specifications or empirical characterisation.

Two-State Constant-Velocity Kalman Filter

Purpose. The two-state Kalman filter extends the position model by additionally tracking the rate of change (velocity) of the measured quantity. It is designed for signals that exhibit persistent trends rather than stationary fluctuations around a fixed value.

Principle. The filter maintains a two-element state vector containing position and velocity. At each step, the prediction phase advances the position estimate by the current velocity estimate, then corrects both using the new measurement. The correction is distributed between position and velocity through two separate gain values, computed from a 2×2 covariance matrix that tracks the uncertainty in each state.

Main Equation. The state prediction is:

$$\begin{bmatrix} \hat{x}_{\text{pos}} \\ \hat{x}_{\text{vel}} \end{bmatrix}_n^- = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{x}_{\text{pos}} \\ \hat{x}_{\text{vel}} \end{bmatrix}_{n-1} \tag{3.11}$$

followed by a measurement-driven correction of both state components using gains K_{pos} and K_{vel} computed from the updated 2×2 covariance matrix.

Parameters. The filter is configured with a diagonal 2×2 process noise matrix **Q** (fields `process_noise[0]` for position noise and `process_noise[3]` for velocity noise), a scalar measurement noise variance R (`measurement_noise`), an initial state vector `state[2] = [init_pos, init_vel]`, and a diagonal initial covariance matrix with `init_cov_pos` and `init_cov_vel` on the diagonal. Off-diagonal covariance terms are initialised to `0.0f`.

Advantages. By explicitly tracking velocity, this filter anticipates trends in the signal and tracks them with lower latency than the position-only model under equivalent noise conditions. It is the most general-purpose filter in the library for non-stationary signals.

Disadvantages. The two-state filter is the most computationally intensive in the library, requiring the full expand of a 2×2 matrix predict–update cycle at each step. It also requires more parameters to tune: both process noise terms and the initial velocity estimate must be chosen appropriately. Memory usage amounts to ten floating-point variables per instance (`state[2]`, `covariance[4]`, `process_noise[4]`, and `measurement_noise`).

Use within the IOBOX. This filter is appropriate for sensor channels that are expected to exhibit systematic trends during normal operation, such as temperature measurements taken during equipment warm-up or cool-down cycles, where the temperature rises or falls at a measurable rate that the position-only model would fail to track accurately.

Filter Manager: Architectural Summary

The Filter Manager provides the IOBOX firmware with a complete, self-contained signal processing toolkit . The choice of algorithm depends on the signal characteristics and the computational budget available: the EMA and difference filters are the lightest options for high-rate channels, while the Kalman filters offer adaptive, statistically grounded noise reduction for channels where noise parameters can be characterised.

3.2.4 Application Layer

The Application layer is the topmost layer of the firmware stack. It does not interact with hardware directly but instead coordinates the Manager modules, orchestrates data flow, and assembles the final output frame. It is composed of four modules: the cooperative task scheduler, the Sensor Module, the Modbus Module, and the Frame Builder.

Cooperative Task Scheduler

Rather than employing a real-time operating system, a lightweight cooperative scheduler was implemented directly in `main.c`. The scheduler is based on a statically allocated array of `TaskEntry` structures, each holding a function pointer (`task_fn`), a period in milliseconds (`period_ms`), and a timestamp of the last execution (`last_ms`). The dispatcher `Application_RunTasks()` iterates over the array on every pass of the main loop and invokes each task function whenever the elapsed time since its last execution meets or exceeds its configured period.

Six tasks are registered in the current configuration, as shown in Table 3.1. All task functions are non-blocking by design: they return immediately if no new data is

available or if their internal period has not elapsed, preventing any single task from starving the others.

Table 3.1: Registered tasks in the cooperative scheduler

Task Function	**Period (ms)**	**Responsibility**
`SensorModule_Task`	10	Poll all enabled sensors
`CAN_Manager_Run`	10	Drain CAN FIFO and update `sysboard.can`
`Modbus_Module_Poll`	20	Process incoming Modbus RTU frames
`EEPROM_StoreFrame`	1000	Persist current `SystemBoard` to EEPROM
`EEPROM_LoadTask`	1000	Reload last stored frame from EEPROM
`FrameBuilderTask`	1000	Assemble and finalise the output `IoFrame`

Sensor Module

The Sensor Module, implemented in `Sensor_module.c`, provides a unified acquisition interface for all enabled sensors. It maintains one private timestamp variable per sensor and, on each invocation of `SensorModule_Task()`, compares the current tick against each timestamp to determine whether the sensor's configured sampling period has elapsed. When it has, the corresponding manager acquisition function is called and the result is written directly into the `sysboard` global structure. The per-sensor sampling periods are defined centrally in `feature_config.h`: 1000 ms for the HTS221TR, 1000 ms for the ENS210, and 1000 ms for the OPT3001DNPR. The module is fully driven by compile-time feature flags; sensors disabled at configuration time introduce no runtime overhead.

Modbus Application Module

The Modbus Application Module forms the bridge between the Modbus RTU stack and the platform data model. It defines a static register map that associates each Modbus register address and object type with the corresponding field in the shared `SystemBoard` structure and an access permission flag. The current register map exposes five read-only input registers: ENS210 humidity at address 0, ENS210 temperature at address 1, HTS221TR humidity at address 2, HTS221TR temperature at address 3, and OPT3001 illuminance at address 4. Floating-point values are scaled by a factor of 100 before transmission, preserving two decimal places within the 16-bit Modbus register format.

Handlers for each supported function code are registered in the Modbus slave

handle and invoked by the Modbus Manager dispatcher upon receipt of a valid request.

Table 3.2: Modbus input register map

Address	**Source**	**Physical Quantity**	**Scale**
0	ENS210 humidity field	Relative humidity (%)	×100
1	ENS210 temperature field	Temperature (°C)	×100
2	HTS221TR humidity field	Relative humidity (%)	×100
3	HTS221TR temperature field	Temperature (°C)	×100
4	OPT3001 illuminance field	Illuminance (lux)	×100

Frame Builder

The Frame Builder is responsible for assembling the final output structure that the IOBOX platform transmits to external systems. Rather than forwarding raw sensor readings directly, the platform encapsulates all acquired data within a structured binary frame whose layout is illustrated in Figure 3.2. This design ensures that any receiving system can unambiguously identify, validate, and decode the payload regardless of which sensors or modules are active in a given deployment.

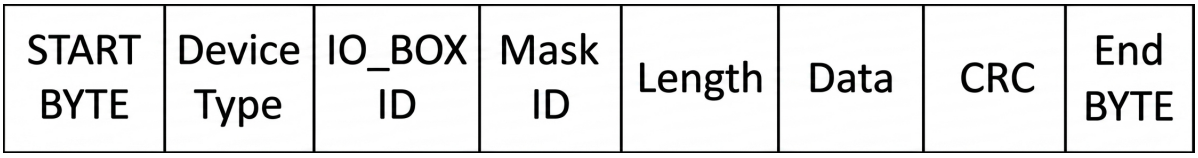

Figure 3.2: Structure of the IOBOX output frame

The frame assembly routine populates each field from the most recently persisted system state, then computes and appends a CRC checksum over all preceding fields. The complete frame format is described in detail below.

Start Byte. A fixed synchronisation marker (`0xAA`) that signals the beginning of a valid frame to any receiver scanning an incoming byte stream, allowing frame boundaries to be established without relying on inter-frame timing gaps.

Device Type. Identifies the category of the transmitting device, allowing a network to host multiple types of embedded nodes simultaneously. A receiving gateway inspects

this field to apply the appropriate decoding logic without prior knowledge of the sender's identity.

IOBOX ID. Carries the unique numeric identifier of the specific IOBOX unit that generated the frame. In multi-node deployments, this field allows a receiving system to associate each frame with its physical origin without requiring any session or connection establishment.

Mask ID. A configuration bitmap describing which sensors and communication modules are active and contributing data to the current frame. Each bit corresponds to one module, as detailed in Table 3.3. The receiver reads this field before parsing the payload to determine the exact byte layout of the Data field, providing both flexibility and forward compatibility.

Table 3.3: Mask ID bit field definitions

Bit	Module	Logic 1 meaning
5	CAN sniffer	CAN data present in payload
4	HTS221TR Temperature	HTS221 temperature present
3	HTS221TR Humidity	HTS221 humidity present
2	ENS210 Temperature	ENS210 temperature present
1	ENS210 Humidity	ENS210 humidity present
0	OPT3001DNPR Illuminance	OPT3001 lux value present

Length. Specifies the total size of the Data field in bytes, making the frame self-describing. The receiver uses this field to locate the CRC and End Byte at their correct positions without re-deriving the payload size from the Mask ID.

Data. Contains the measurement values and communication data collected during the current acquisition cycle. Its contents are governed entirely by the active bits in the Mask ID field. In the current configuration, the payload includes temperature and humidity readings from both sensor units, the OPT3001DNPR illuminance value, and the CAN frame payload comprising the message identifier, RTR flag, data length code, and up to eight data bytes.

CRC. A 16-bit integrity checksum computed over all preceding fields using the CRC-16/IBM polynomial. The receiver verifies this field before consuming any measurement

data; a mismatch indicates a transmission error or a corrupted frame and requires the frame to be discarded.

End Byte. A fixed termination marker (`0x55`) that constitutes a second layer of frame boundary validation. Together with the Start Byte, it allows the receiver to detect framing errors — for example, those caused by a corrupted Length field — even when the CRC alone might not reveal the misalignment.

The complete field layout is summarised in Table 3.4.

Table 3.4: IoFrame field summary

Field	**Value**	**Description**
Start Byte	`0xAA`	Frame synchronisation delimiter
Device Type	Compile-time constant	Node category identifier
IOBOX ID	Compile-time constant	Per-unit unique identifier
Mask ID	6-bit bitmap	Active module bitmap
Length	Variable	Data field length in bytes
Data	Variable	Active measurement payload
CRC	CRC-16/IBM	Integrity check over all preceding fields
End Byte	`0x55`	Frame termination delimiter

The frame assembly routine retrieves the last persisted system state and fills every field sequentially before computing and inserting the CRC. This design ensures that the output frame always reflects the most recently stored platform state, providing continuity across power cycles and making the frame contents auditable through the non-volatile storage layer.

System Data Flow

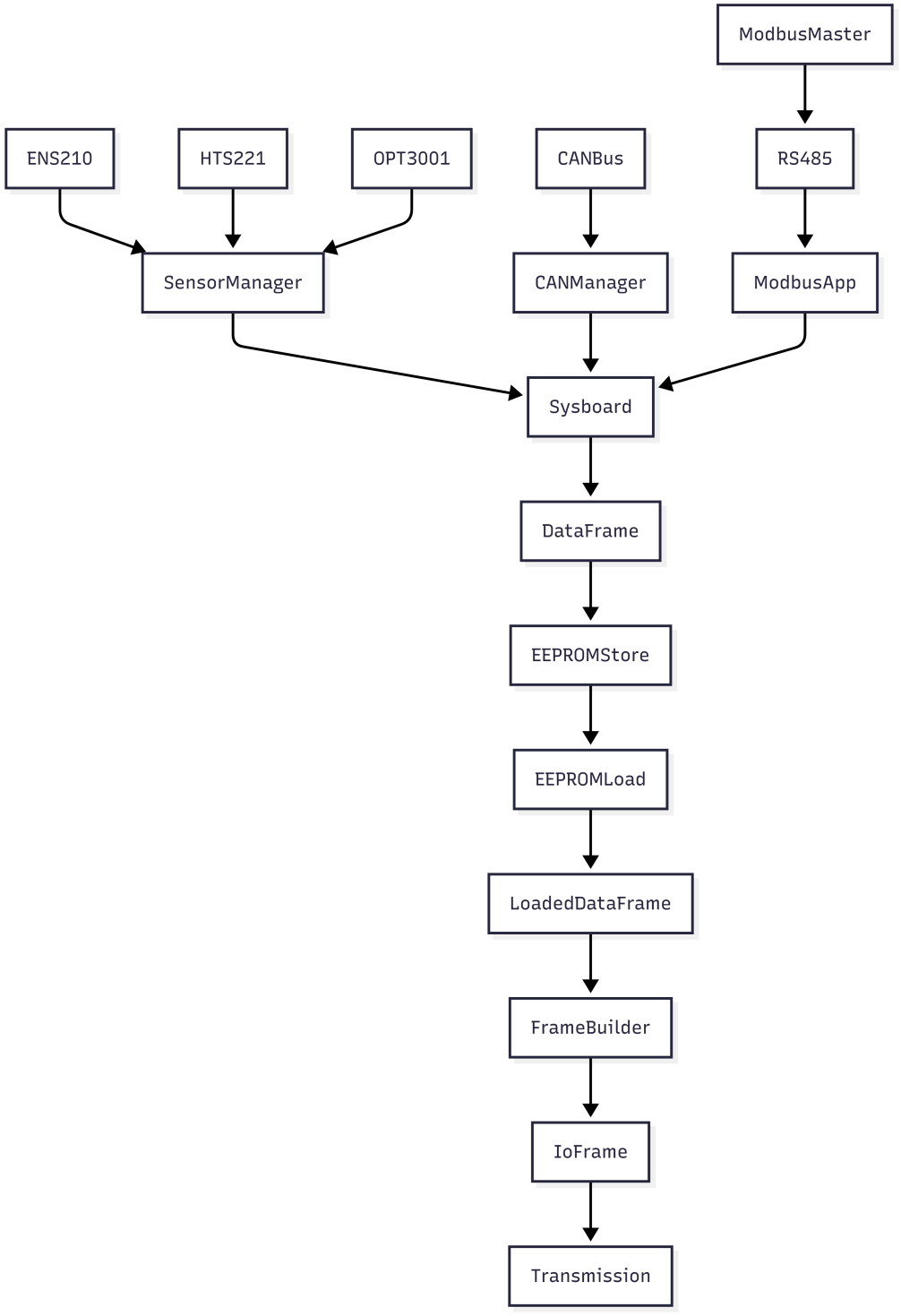

Figure 3.3: End-to-end data flow across the three firmware layers

The complete data path through the system can be summarised as follows. Physical sensor measurements are acquired by the Sensor Module and written into `sysboard`. Simultaneously, CAN frames captured by the CAN Manager are stored in the same structure. At every 1000 ms interval, the EEPROM Store task serialises `sysboard` into a `DataFrame` and writes it to non-volatile memory; the EEPROM Load task immediately retrieves the last valid frame. Finally, the Frame Builder consumes the loaded `DataFrame` and produces the `IoFrame` output, ready for transmission. Throughout this pipeline, any Modbus master connected via RS485 may read live sensor values directly from `sysboard` through the Modbus Application Module at any point in the cycle.

The software architecture described in this section provides a clean separation of concerns, compile-time configurability, and a deterministic execution model without the overhead of a real-time operating system. The following section presents the validation results obtained on the target hardware.

3.3 System Validation and Functional Testing

Each hardware interface was tested separately before any integration work began. The goal was simple: make sure every peripheral actually works before building anything on top of it. Tests were run directly on the STM32G474MBTx target board using STM32CubeIDE, with results printed through the SWO debug terminal to ensure a visual validation for evrey test .

3.3.1 ADC Validation

ADC channels were sampled in a single scan sequence to check that the conversion phase was set up correctly. The results were printed to the terminal as shown in Figure 3.4.

Channel 0 read zero, which was expected that pin was left unconnected. Every other channel returned a value somewhere in the 0–4095 range as expected .The scan ran without errors and the results looked physically reasonable, so the ADC driver was considered validated at that point.

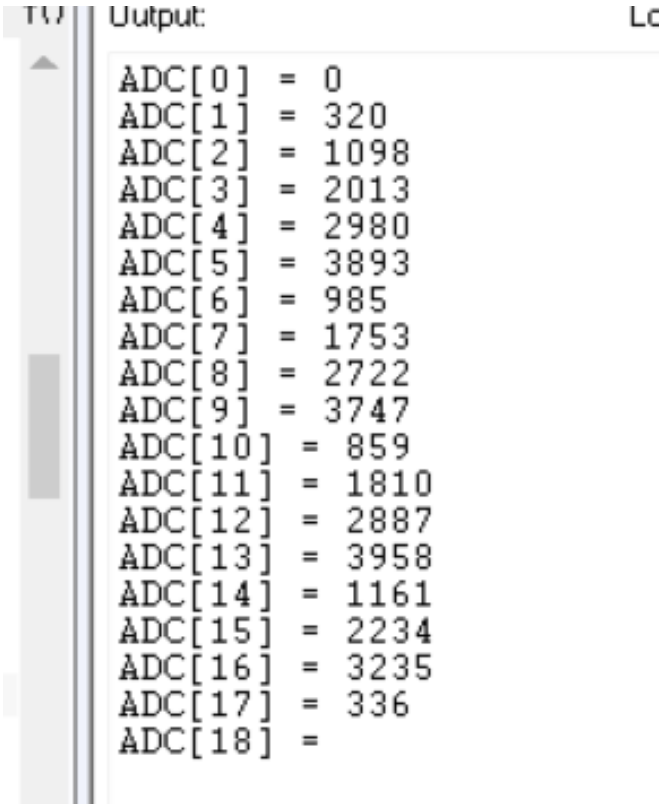

Figure 3.4: ADC raw output across 19 channels

3.3.2 SPI Communication Test

The SPI driver was tested by reading data from an esp32 SPI peripheral and printing each received byte to the IAR SWO debug terminal. Seven consecutive receive frames were captured each containing eight bytes as being sent from the esp32 (Figure 3.5). Clock polarity, phase, and bit order were all set and the received data matched the expected response format.That was enough to confirm the SPI bus was working.

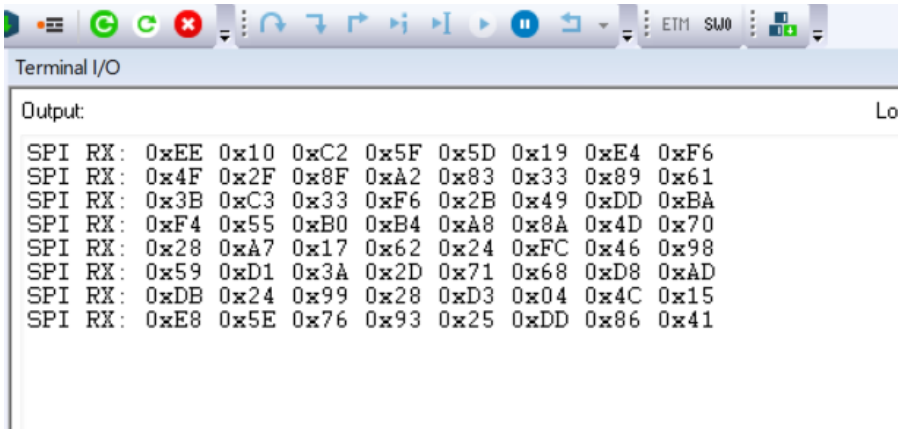

Figure 3.5: SPI receive frames captured on the debug terminal

3.3.3 CAN Standard Identifier Test

A CAN GPS module was connected to the board with a CAN transceiver and a 120 ohm resistor and configured to send frames with standard 11 bit identifiers at a fixed rate. The firmware was succeeded to print each received frame payload to the terminal.

Figure 3.6 shows the captured output. Three payload patterns cycled in a fixed sequence throughout the test as shown in the figure bellow so The part of the payload `DE AD BE EF` stays identical across all of them which confirms the payload was not getting corrupted on reception. The frame cycle matches exactly what was sent from the CAN GPS module side meaning our CAN module works perfectly.

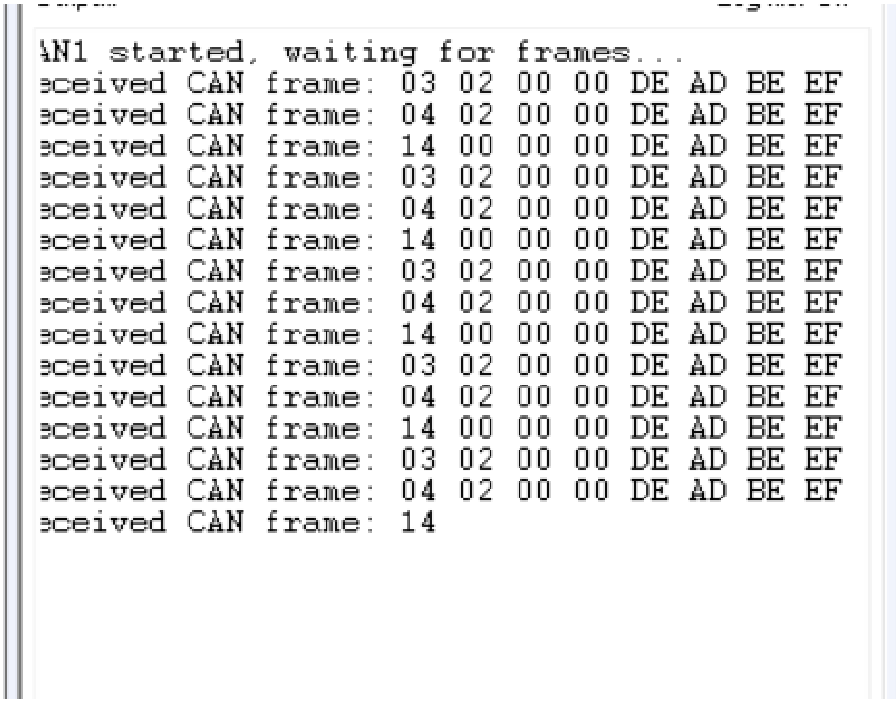

Figure 3.6: CAN frames received with standard 11-bit identifiers

3.3.4 CAN Extended Identifier Test

The same test was repeated using 29-bit extended identifiers to make sure the CAN filter was configured to accept that frame format as well.

in the test we configured the module to show evrey part of the frame not just the payload part and all received frames show the identifier `0x1752286` with `DLC=8`, as seen in Figure 3.7. The first two bytes of the payload increment across frames — `C6 14`, `C7 14`, `E5 14` which matches the counter field being incremented as we expect from the sender . The last four bytes stay fixed at `DE AD BE EF` as before confirming no corruption. The firmware printed the full 29-bit ID correctlyso the extended frame mode was working as expected.

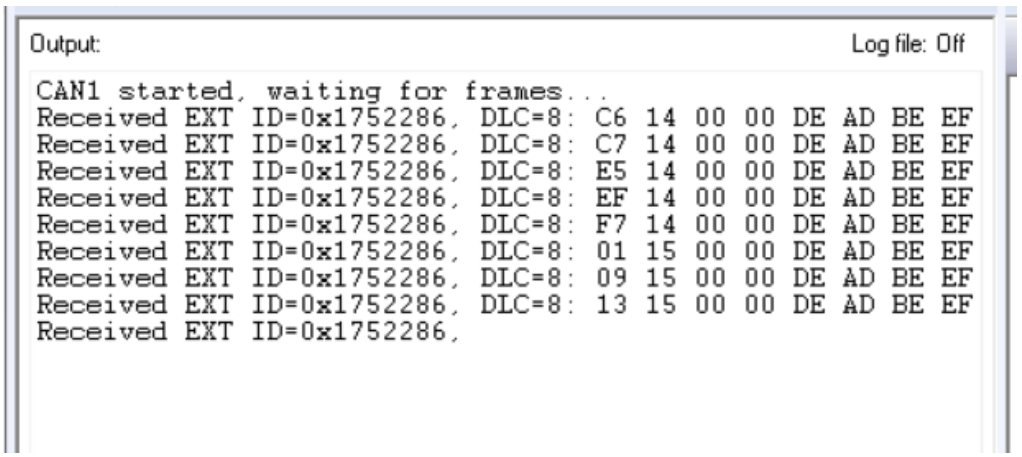

Figure 3.7: CAN frames received with extended 29-bit identifiers

3.3.5 ENS210 Humidity Validation

The ENS210 was tested by running the acquisition routine in a loop and watching the humidity output on the terminal. The sensor sits on I^2C multiplexer channel 2.

The first value printed was 0.00 %RH, which happened because the reading was taken before the sensor finished its first measurement cycle . the sensor was configured in single-shot mode. After that readings settled between 53.29 and 53.39 %RH which is a reasonable indoor humidity level for the conditions during testing. A wet paper was then held close to the sensor and the readings climbed to 83.59 %RH before dropping back down once it was removed. Figure 3.8 shows the full result. The sensor responded to the change the multiplexer channel selection worked and the conversion gave physically corrected numbers.

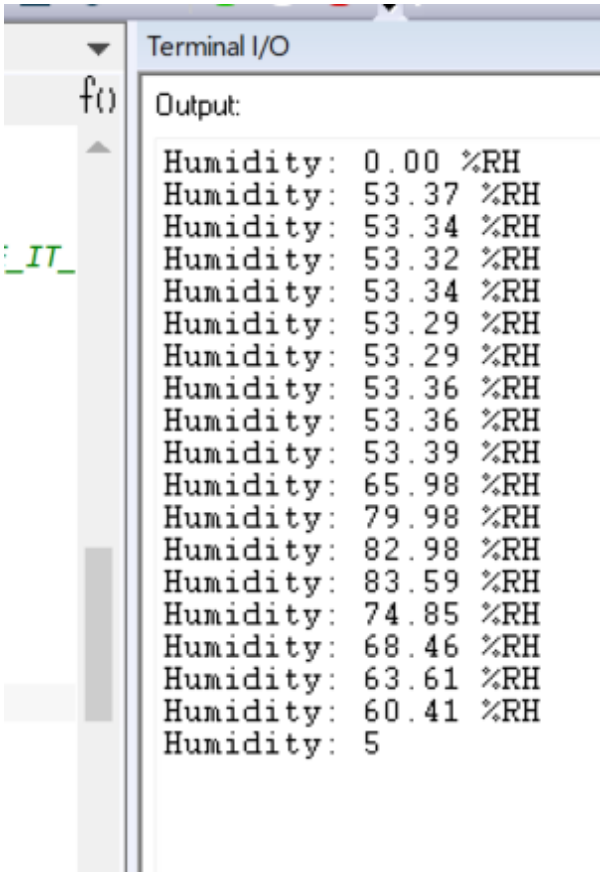

Figure 3.8: ENS210 humidity readings during ambient and induced humidity test

3.3.6 ENS210 Temperature Validation

The temperature output of the ENS210 was verified alongside the humidity test, using the same acquisition loop on multiplexer channel 2.

The first reading came back as -45.00 degreeCelsius, which is the value the conversion formula produces when the sensor has not yet completed its first measurement the raw register still holds its reset state at that point. From the second reading the output stabilised, cycling between 38.6 degreeCelsius and 39.96 degreeCelsius as shown in Figure 3.9. That range is slightly above typical room temperature because the sensor sits close to active components on the board and picks up some of the heat they dissipate. which confirms that the sensor is running correctly and that the temperature conversion is being applied successfully .

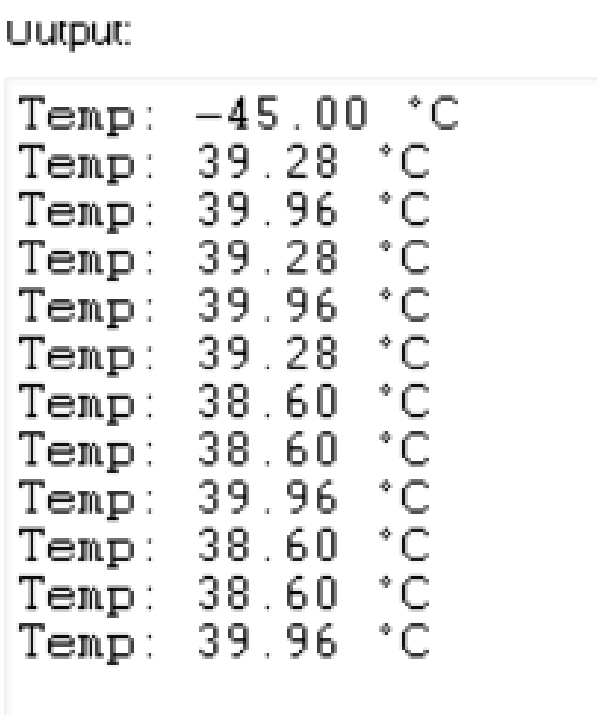

Figure 3.9: ENS210 temperature readings showing stabilisation after sensor warm-up

3.3.7 OPT3001 Sensor Validation

The OPT3001DNPR was tested by running acquisitions while changing the light level above the sensor and checking that the lux output responds to the changes correctly.

Figure 3.10 shows the raw register values alongside the computed lux results after conversion . Lux readings ranged from 63 to 493 during the test, which corresponds to the actual lighting level in the room lower values when a finger was placed over the sensor higher values when a lamp was brought closer. which validates our work with the adressing , the conversion and the good calibration .

```
Raw: 0xD0EE, Lux: 338.00
Raw: 0x23D0, Lux: 117.00
Raw: 0x5011, Lux: 139.00
Raw: 0x2B05, Lux: 134.00
Raw: 0x98F2, Lux: 133.00
Raw: 0x02F7, Lux: 339.00
Raw: 0xEF65, Lux: 63.00
Raw: 0x1930, Lux: 334.00
Raw: 0x9632, Lux: 116.00
Raw: 0x7370, Lux: 202.00
Raw: 0x38EE, Lux: 96.00
Raw: 0xA8E7, Lux: 194.00
Raw: 0x5E1E, Lux: 234.00
Raw: 0xE763, Lux: 69.00
Raw: 0x24AD, Lux: 89.00
Raw: 0xC6F3, Lux: 493.00
Raw: 0xA611, Lux: 69.00
Raw: 0xE27B, Lux: 360.00
Raw: 0x385B, Lux: 352.00
Raw: 0x31FD, Lux: 111.00
Raw: 0x4961, Lux
```

Figure 3.10: OPT3001 raw register values and computed lux output

3.3.8 HTS221TR Sensor Validation

The objective of this test was to confirm that the HTS221TR driver was correctly acquiring temperature and humidity data and exposing it through the communication interface.

Figure 3.11 shows the captured data with no missing response or errors appears across the entire captured sequence, confirming that the sensor data was being read and transmitted correctly.

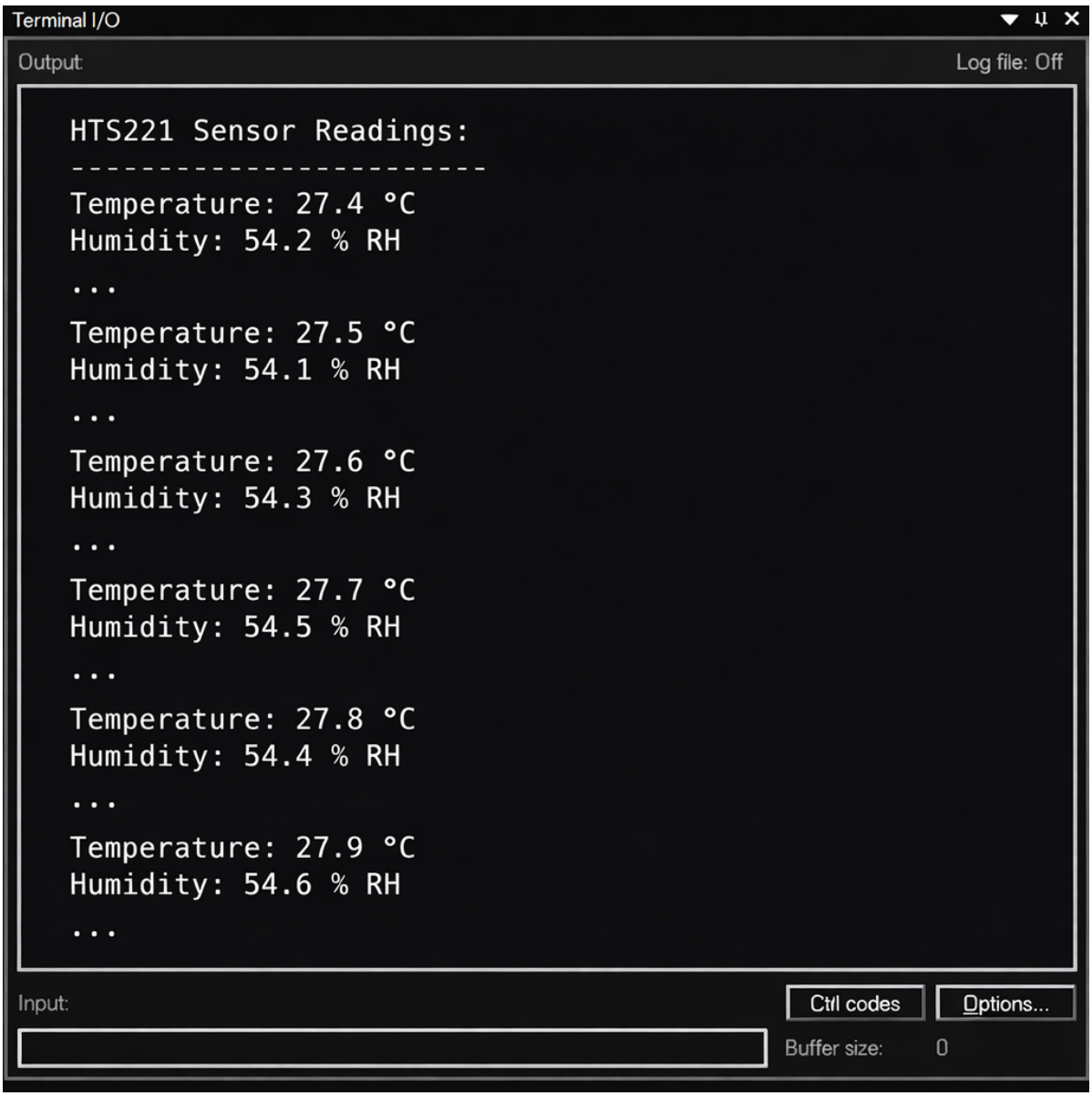

Figure 3.11: Communication traffic log during HTS221TR sensor validation

3.3.9 RS485 Communication Test PC Side

The RS485 link was tested by connecting the test board to a PC running the serial terminal over COM6 at 115200 bps. The STM32 transmitted sensor data periodically over the RS485 bus and the PC terminal was used to verify reception.

Figure 3.12 shows the output on the PC side. After the USART3 initialisation message and the RS485 node initialisation confirmation, the board started transmitting

temperature and humidity readings. The PC received temperature and humidity values

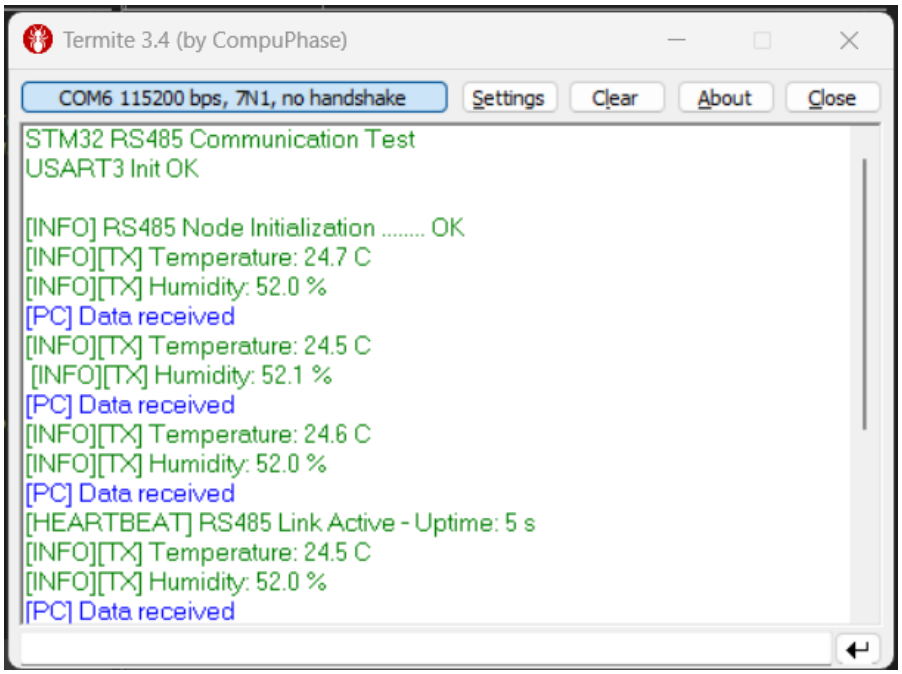

Figure 3.12: RS485 reception output on the PC terminal (Termite 3.4)

3.3.10 RS485 Communication Test — STM32 Side

The same RS485 test was done from the STM32 side using the STM32CubeIDE debug terminal, to confirm that the microcontroller was correctly handling both the transmission and the reception .

Figure 3.13 shows the STM32-side output. The sequence starts with the same initialisation messages then logs each transmitted temperature and humidity values . meaning transmit and receive mode was working correctly.

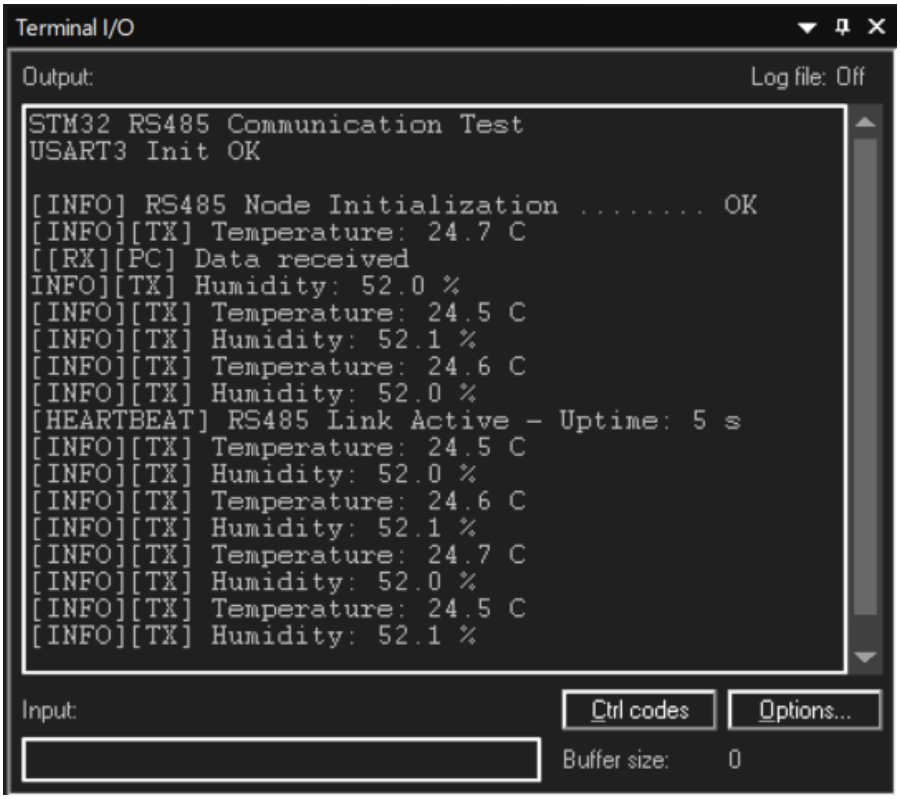

Figure 3.13: RS485 transmission log on the STM32 debug terminal

3.4 Conclusion

BIBLIOGRAPHY

[1] ISIMM. consulted on 04/13/2026. http://www.isimm.rnu.tn/public/.

[2] BWS. consulted on 04/13/2026. https://bewireless-solutions.com/en/about-bws/.

[3] VS code. consulted on 04/13/2026. https://code.visualstudio.com/.

[4] IAR EW. consulted on 04/13/2026. https://www.iar.com/.

[5] STM32CubeMX. consulted on 04/13/2026. https://www.st.com/en/development-tools/stm32cubemx.html.